



Service-Oriented Programming Paradigm

15

15.1 Towards Services

15.1.1 Distributed Programs

In Chap. 14, we saw how distributed programming is a kind of concurrent programming where concurrent programs run on different distributed physical structures.

Physical distribution is the key to understanding how the practice of building distributed applications compelled the definition of a dedicated programming paradigm, oriented towards “services”.

Distributed programming has made it ordinary (as much as we take these as commodities) to exchange media (messages, photos, videos) with people all over the world, as well as accessing, ordering, and paying for products and services online. However, the practice of programming distributed systems comes with its specific challenges. Indeed, the departure from shared-memory models and the availability of solutions that let any two processes “talk” to each other (briefly presented in Chap. 14) makes it difficult to check the correctness of distributed programs (e.g., that the composition is type-safe).

Looking at a practical case, *local* programs—we use the term throughout the chapter to indicate non-distributed applications—let us check Data Abstraction (see Chap. 9) by assembling the program modules and verifying that said modules comply with the interfaces they promise to implement. This holds because we can check compliance over the whole program (either statically or dynamically, cf. Sect. 8.8). That assumption generally falls for distributed programs. As seen in Sect. 14.5, one can easily parametrise the different techniques that let processes exchange messages, so that any two processes can dynamically interact and exchange messages. In this dynamic compositions, the only things that the processes witness are each other’s messages, which makes checking compliance a multi-faceted problem:

- Are messages transported in a reliable or unreliable fashion?
- Do the programs see the same ordering of messages (e.g., as seen by the sender)?
- Can the receiver re-order messages?
- Are the messages in the format that the addressee expects to see (is it able to parse them)?
- Do the messages invoke operations that the addressee actually provides?
- Is the data contained in the messages compliant with the types of the related operation?
- How should the sender and receiver handle errors (do they need to communicate them to the other party)?

In the previous chapters, we saw techniques for local programs that can give us (static or dynamic) guarantees on compliance—and one can assume invariants, like the preservation of invocation order between caller and callee, because they share the same runtime. In distributed programs, caller and callee not only can run on different machines, but communication protocols determine how they exchange messages. Moreover, they can also run different runtimes, with different ways to represent the same concepts and data structures (e.g., when implemented using different programming languages). This means that, to provide compliance guarantees, linguistic techniques must either assume some of those elements fixed (e.g., communication preserves order, errors work like exceptions, cf. Sect. 7.3) or be able to compositionally reason on these.

As a concrete example, we can write a `server` that waits on a port for a message from some `client`. This exchange represents a dynamic module composition of which we cannot check any static compliance (we do not know the identity of the `client` until it contacts the `server`). Of course, if the `server` receives an unexpected message from the `client` (e.g., on an operation it does not provide) the `server` might infer that the `client` does not comply with the interface it agreed to implement. However, this check (if any) is local only to the `server`, and, potentially, it does not make sense to inform other modules of such occurrence, e.g., even informing the `client` might not be possible, unless there was a previous agreement (an interface) that the `client` is supposed to implement for such occasions.

These challenges, called for the development of a discipline, ascribed to the umbrella term “service-oriented programming”, equipped with dedicated abstractions, techniques, and technologies able to help programmers define, reason on, and verify properties of the composition of distributed programs.

15.1.2 Open Systems

From a historical perspective, distributed programming exploded in popularity in the 1990s, with the advent of the digital economy—based on the exchange of digital information among people and businesses—and thanks to the widespread availability of networked connections and the Internet—the global system of interconnected computers behind, e.g., the World Wide Web and electronic mail.

To reach worldwide adoption, many developers of distributed applications needed to follow an open approach to distributed programming. Their users needed interoperating programs to allow them to interact and communicate. While one can achieve interpretability by adopting a closed approach—where the components of a given program are fixed, as well as their runtimes; which has the benefit of supporting compliance checks akin to local programs—this would hardly scale to meet the ever-growing needs of millions (and then billions) of users spread across the globe. For example, this approach would imply the unreasonable need to update all programs (in all computers spread around the globe) that interact over the Internet, just because we introduced or updated a single component, e.g., a website.

Thus, to not stymie the growth of the Internet, developers needed to adopt an open approach to implementing distributed programs. In particular, they needed to define and adopt standards that allowed their software components to interact with other components they neither wrote nor knew any implementation details of.

As an example from our daily lives, we assemble an open, distributed system every time we connect the Web browser installed in our devices to a Web server that provides access to information pages, seller listings or search results. Among the many standards our Web browsers are built upon, we find the Internet Protocol (IP), the Transmission Control Protocol (TCP), the Hypertext Transfer Protocol (HTTP), the Extensible Markup Language (XML), and the JavaScript Object Notation (JSON).

15.1.3 Loose Coupling and Interoperability

Summarising, a relevant portion of distributed programs are and need to be open. The two fundamental principles that support such openness are *loose coupling* and *interoperability*.

Loose coupling indicates a weak association among the modules that make up a program, so that internal changes to a module are unobservable by the other modules in the program. In a distributed setting, this principle—which extends the principle of abstraction seen for objects (see Sect. 10.2.3)—means that modules have little to no knowledge of the internal details of the other modules and, thus, a high degree of freedom concerning the technology used for their implementation (e.g., hardware, execution environment, implementation language, and logic).

The other piece of the puzzle regards interoperability. We have loosely-coupled modules that can employ disparate technologies for their implementation. Now, to realise the logic of our distributed program, we need to let these modules invoke operations of and exchange data with the other modules that make up the program. Coupling and interoperability are linked concepts. Strongly-coupled modules are inherently interoperable (e.g., imagine an object invoking a method of another object, sending to it the reference to some data in memory). This is because strongly-coupled modules implicitly share a large amount of interoperability choices dictated by the implementation technology, e.g., how modules interact or how data is represented, moved, and stored. On the contrary, loosely-coupled modules need to make interoperability choices explicit, e.g., how one module can invoke an operation of another

module (e.g., what transport technology) and how they should transmit data (e.g., what data format).

While interoperability decisions can concern a single distributed program—so that the developers can adopt internal conventions to make their modules interact—to obtain open distributed systems, we need to define (and adopt) standard technologies for interoperability. In this way, developers can integrate into their distributed programs any modules with interoperable technology stacks. Similarly, different distributed programs can interact as long as they are interoperable.

When looking at the linguistic abstractions and technologies for service-oriented programming, we will see how these are involved in capturing and supporting loose coupling and interoperability of services.

15.1.4 Approaching Services

Although the first proposals for a service-oriented paradigm date to the early 2000s, the concept of *service* in computer science predates service orientation. Indeed, we can find the name “service”, associated with some of the properties that service-oriented programming ascribes to it, in contexts that precede the paradigm.

For instance, we find many examples of network technologies characterised by the interaction of layers, each devoted to dealing with a specific aspect of communication. Conceptually, developers of network stacks see these layers as abstract machines, each providing these functionalities as services to the adjacent layers. For example, a layer can manage the physical hardware that emits and receives signals, and it can serve two operations; one that returns an array of bits once it received a set amount of signals and one that accepts an array of bits, with the promise to emit them as a sequence of signals. An adjacent layer to the one just described can use this “physical” service to implement some packet-switching logic.

Similarly, in operating systems, we find (*nomen omen*) Windows services and Unix daemons (not to be confused with daemon threads, cf. Sect. 14.7.2) that are processes run by the operating system to provide services to users. An exemplary Unix daemon is `sshd`, which is a program that provides operations that let users perform remote access and command execution. Another is `dbus-daemon`, a component of the reference implementation of the D-Bus inter-process communication service, which provides operations that let processes communicate with each other as if they were connected via a communication bus—the daemon routes messages from and to any process connected to the bus.

Remembering the contents of Sects. 9.2 and 10.2.3, this tendency towards services does not surprise us at all; it is the application of the concepts of information hiding and data encapsulation, where we break down the complexity of a software system into modules—each exposing a set of operations to the other modules and hiding away details like internal logic and state representation.

15.2 Elements of Service-Oriented Programming

15.2.1 Services, a Layered View

Despite the existence of conventions and practices for building distributed programs out of software units seen as services, it is only with the advent of service orientation that services became the first-class citizens of a dedicated programming paradigm.

In service orientation, a *service* is an independent process offering operations accessible via message exchange. This means that the service hides its internal state (and representation) and the only access surface for users is the one provided by operations, accessible through message-based invocations.

Services and Objects

If the above definition of service reminds some readers of the definition of objects given by object orientation, they are not on a wrong track. Indeed, as stated in Sect. 10.2.1, an object is a capsule that encloses both some data and the operations that manipulate it and the users of the object can interact with the former only by invoking those operations, collected in a public interface, via messages.

Conceptually, the distinction between objects and services is relatively thin, as it hinges on the fact that an object does not correspond to an independent process. However, this small conceptual divergence makes a lot of difference in practice.

As seen, e.g., in Sect. 10.3.1, for performance reasons, the mainstream interpretation of “invocation via messages” in object-oriented languages concretises in efficient procedures that execute the code related to a given method/operation and pass to it possible parameters as memory references.

If, on one side, this interpretation makes implemented object-oriented languages efficient, on the other side, it makes it impossible for programmers to natively let objects interact with each other via other means than memory—resorting to either implementing ad-hoc procedures, such as stubs for remote procedure calls (cf. Sect. 14.5.4), or using technology-specific solutions, like Java remote method invocations (cf. Sect. 14.7.3).

The principle of independence of execution of services breaks any of such technology-specific optimisations, and it forces developers (and language designers) to first consider the conceptual traits that determine a service and, as a consequence, the stack of technologies that one can use to implement it.

We draw the relation between the conceptual and the technological layers of a service in Fig. 15.1. In the figure, the two-dimensional positioning of the items follows an arrangement that keeps close elements that relate to each other between the layers (where, of course, the technological one is not exhaustive).

Conceptual Layer

The conceptual layer describes in abstract terms the traits that characterise a service, which then identify different sets of technologies that one can use to implement those traits and, by extension, the service.

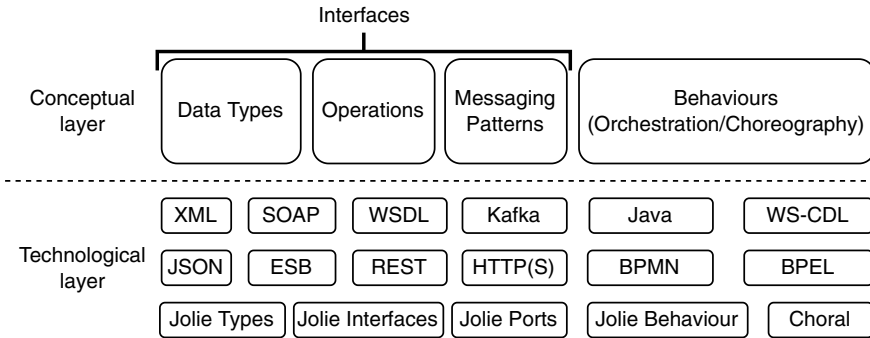


Fig. 15.1 Layers of service-oriented programming

As shown in Fig. 15.1, we can roughly divide the elements of the conceptual layer in two sets. One is that of the *interfaces*, where we find the items that the service exposes to its users: the operations it is willing to perform, how users should call them (i.e., their respective messaging patterns), and the shape of the data that it expects to receive and yield. The other set is that of *behaviours*, which regards the logic of the service both used internally, e.g., to compute the response following a request from the user, and to perform external invocations, e.g., to other services to accomplish some multi-party goals.

Interfaces

Like in abstract data types and objects, interfaces represent the public view provided to users. In interfaces, the “data types” describe the structure of the data that the service either expects to receive and/or promises to produce at the invocation of its operations. The “operations” offered by a service correspond to a set of invocable labels. Notably, services do not commit to the typical request-response “messaging pattern” seen for, e.g., object methods (which, in turn, adheres to the common stack-based control-transfer model, discussed in Chap. 7). On the contrary, besides being associated with one or more data structures used for the invocation, operations specify the way in which they expect to exchange messages with their users. For example, an operation invocation can follow the unreliable one-way message exchange pattern, where the invoker sends to the service its message without any guarantee of either reception nor consumption from the service. Another alternative are reliable one-way exchanges, where the invoker waits to receive an acknowledgement of message reception (but not of its consumption) from the service.

Behaviours

The last element of the conceptual layer, that of *behaviours*, ties together state and interfaces of a service.

To introduce this concept, let us look one last time at non-concurrent models for objects. While objects enclose state, the latter does not influence the possibility for

the object itself to respond to an invocation on any of its methods—at most, we might trigger some errors or exceptions, but the invocation is always available. In services, a developer can write a service in a way that it expects to receive a message for, e.g., an `access` operation, which enables the invocation of a `count` operation—that returns to the user the current count of `accesses` registered by the service. As argued in Sect. 14.5.3, if the user is not aware of that linearisation of the two operations, their interaction could end up in a deadlock. For example, the user can send in parallel the two invocations, assuming that the service would first consume the `access` and then (consume and) respond to the `count`. On the contrary, if *a*) due to non-determinism, the service can receive first the invocation of `count` and then `access`, and *b*) the service assumes a FIFO (cf. Sect. 14.4.2) ordering of the messages—so that it consumes the messages only in their order of arrival—the interaction between the two becomes stuck.

For behaviours, service-oriented programming distinguishes between two approaches to implement the distributed logic of services: *orchestration* and *choreography*. The plural “services” in the previous passage is important, as the matter on *how* to program the logic of a service becomes relevant when a single service might not suffice to accomplish some task, but rather requires the interaction of two or more services.

As an example, imagine a `shop` and a `bank` services, which need to interact with a `user` client, to implement an e-commerce transaction. There, the logic of the transaction can either be controlled by one of the two services or distributed among them.

In orchestration, one service, called the *orchestrator*, implements the distributed behaviour by coordinating the other services that take part in the composition. In the example, if we elect the `shop` as the orchestrator, it would, e.g., coordinate the interaction with the `user` and the `bank`, such as asking the user for its address, and preparing the transfer order for the `bank`.¹ Here, the `user` and the `bank` assume *passive* roles, i.e., they expose basic operations that are (ideally) always available and that the orchestrator (the `shop`) invokes to enact the distributed logic of the transaction. Summarising, in orchestration, the distributed logic is mainly defined and executed by the orchestrator, which coordinates and mediates all the interactions among the other services that take part in the composition.

While orchestration has the good trait of concentrating the logic of the distributed composition in one service—making it easy for programmers to both implement and check its correctness against some reference specification—the orchestrator is a single point of control. This can present both functional and performance problems. For instance, the orchestrator can see all the data that it routes among the services it coordinates, and it can unilaterally dictate what operations the other services must perform. If our program has a functional requirement of not allowing the parties to trust the central orchestrator to do the “right thing”, we cannot adopt orchestration—

¹ For simplicity, we assume a naïve `bank` that directly trusts third-party requests and does not employ authentication procedures, like multifactor authentication, to make sure requests are genuinely coming from its clients.

imagine, in the user-bank-shop example, a fraudulent shop that has the bank bill two times the user for the same product. Looking at performance, orchestration can imply an increase in the number of messages exchanged among the services. For example, since the shop orchestrator mediates the interactions between the bank and the user, instead of one direct message, from the former to the latter, we have three: (1) the shop asks the bank to provide the message for the user, (2) the bank responds with the message, and (3) the shop sends the message to the user. When bandwidth is limited, it has high latency or message payloads are heavy, this increase in the number of messages exchanged can severely impact the performance and usability of the distributed program.

Hence, there are scenarios where we cannot clearly find an orchestrator to mediate the whole distributed interaction. In this case, the services must implement the distributed logic of the transaction as a choreography, i.e., in a way that each participant knows and implements its part in the interaction. The name “choreography” comes from the fact that, like in an actual choreographed performance, each service in a choreography plays a specific role in it and enacts the actions which that role is supposed to carry out. We can see the duality between orchestration and choreography as the difference between, respectively, having one service enacts the logic of the whole distributed program by coordinating the other services and having the services enact that logic through their autonomous interactions. In this case, the *choreographer* is not a service itself (as opposed to the orchestrator), but it is the entity (e.g., the programmer) that defined the choreography.

Choreographies, although not named in this way, predate service orientation and are akin to any model useful to specify the sequence of interactions among different parties (some early examples in computer science are the security protocol Alice-and-Bob notation, message-sequence charts, and sequence diagrams).

While the general understanding of orchestration is established, there is currently no standard interpretation of the usage of choreographies. For instance, we can see choreographies as reference specifications, so that programmers should look at a choreography to correctly implement the services that participate in it. Another interpretation is that of an advanced kind of type, where programmers use choreographies to mechanically check that the implementation of their services complies with their intended role in the choreography. A third interpretation is considering choreographies as programs. In this case, developers specify the actions and interactions of all the involved services as a choreographic program. Then, given a source choreography, the developers use a compiler to automatically generate the correct code of all the services that participate therein. We illustrate these possible interpretations in Sect. 15.2.5. Of course, these interpretations are far from being mutually exclusive and, for example, one can mix choreographic programs and types, e.g., to partially implement a part of a choreographed system and define the remaining parts as typed parameters that the user can provide at a later stage.

Technological Layer

The technological layer comprises solutions that deal with one or more of the elements that characterise a service at the conceptual level. Figure 15.1 shows a non-

exhaustive list of these technologies, which we cover in the next sections. Since our interest, here, is the relation of these solutions with programming languages, the following sections do not aim at being comprehensive in covering any of the presented alternatives, but rather focus on the relevant concepts that mark one or more of these from the point of view of language constructs, abstractions, analyses (e.g., type checking), and semantics (e.g., runtime).

15.2.2 Data Types

The arguments presented for data types in Chap. 8 hold also for service orientation. What is peculiar about service orientation is that the types and values presented to outside users usually differ from the types and values internally defined and handled by the behaviour of a service.

This discontinuity emerges from the fact that, to allow data to flow between distributed processes, we need to transform it from its internal, in-memory representation in a service into a format amenable to transmission—with a procedure called *serialisation*—and, upon reception, to translate it back—with a complementary procedure called *deserialisation*—into an in-memory representation. Of course, a good data format (with its related translation procedures) should not be tied to a specific language runtime or hardware architecture and guarantees the transfer of the semantically-relevant elements of a given data structure among services.

To make a concrete example, if service A wants to send even a basic value, such as an integer, to service B, it cannot do that by directly copying the bits found in its memory for that value. Indeed, A does not know any implementation or architectural details of B, and it cannot reliably count on the fact that B's memory layout could host (let alone faithfully represent the data in) its memory layout.

As expected, these concerns appeal to the principles of loose coupling and interoperability of interacting, distributed processes.

Hence, when talking about the data types that characterise the operations of a service, we essentially talk about two factors: the structure of the data in transit and the data formats used to serialise it.

While formats vary, a common trait of many of these is that the structure of data is akin to records with tree-like, acyclic structures—mainly dictated by the complexity of reliably representing and (de)serialising memory references.

Technology-wise, we find solutions generally partitioned between human-readable and binary formats. The former serialise data into sequences of printable characters that humans can easily read and are unambiguously parsable by machines. The latter forgo (human) readability and encode data into strings of bits that optimise memory occupation and/or transmission.

Here, we use as concrete examples two of the most popular human-readable formats employed in the context of service-oriented programming: the JavaScript Object Notation (JSON) and the Extensible Markup Language (XML).

As its acronym suggests, JSON finds its roots in JavaScript—a programming language originally introduced for web-page scripting—, however, the format is

independent of its inspiring language, from which it draws its basic data types and the syntax to represent structured data.

The basic data types of JSON include booleans (represented with the symbols `true` and `false`), (double-precision, floating-point) numbers, and strings (any sequence of UNICODE characters). Composite data structures include arrays that are heterogeneously typed (the format does not enforce elements of arrays to have the same type), and objects, which are collections of key-value pairs, where keys are strings (similar to record types). As an example, we report below a relatively complex data structure to show the mixing of composite and basic data types in JSON structures.

```
{
  "name": "Argo",
  "crew": [
    { "name": "Jason",      "role": "Captain" },
    { "name": "Heracles",  "role": [ "Mastman", "Ballast" ] },
    { "name": "Tiphys",    "role": "Helmsman" },
    { "name": "Atalanta", "role": [ "Pitman", "Ballast" ] },
    { "name": "Argos",     "role": [ "Shipwright", "Ballast" ] }
  ],
  "need repair": false,
  "cargo hold": 5,
  "cargo": [ "goat", "golden fleece", "dragon's teeth" ]
}
```

Above, we define an object that represents a ship, whose elements are enclosed within the outermost pair of curly brackets. Elements include the name of the ship (a string with the characters “Argo”) and a `crew` array (enclosed within square brackets) that contains a sequence of objects representing the crew members with their name and role (either a string or an array of strings, in case the member has more than one role on the ship).

XML adopts an even more minimalistic take on basic types than JSON: any piece of XML data is a string of characters. However, the format gives the possibility to developers to constrain and validate XML-formatted data through the definition of data types carried (or referenced) within the data itself.

Much like JSON, also XML has a “father” model, as it draws inspiration from the Standard Generalised Markup Language (SGML), a document-sharing language standardised in the 1980s—for perspective, XML dates to 1995 and JSON to the early 2000s.

Like JSON, also XML describes data as tree-shaped documents, where tags represent the data structure and carry within them the string-encoded data. For instance the line `< name > Argo < /name >` is part of a document with a tag called `name` (non-empty-element XML tags are always in the form `start-tag < TAG_NAME > end-tag < /TAG_NAME >`) that carry the string value `Argo`. On top of this, XML documents can carry headers that describe the formatting of custom data types, called Data Type Definitions (DTD), which programmers can use to restrict the values that a tag can hold—e.g., by defining the content of the tag as a floating point number. For completeness, we report below a possible XML-formatted translation of the JSON-formatted data structure shown above (notice in particular how arrays derive from ordered repetitions of the same tag, e.g., `< crew >< /crew >` and `< role >< /role >`).

```

<ship>
  <name>Argo</name>
  <crew>
    <name>Jason</name> <role>Captain</role>
  </crew>
  <crew>
    <name>Heracles</name> <role>Mastman</role> <role>Ballast</role>
  </crew>
  <crew>
    <name>Tiphys</name> <role>Helmsman</role>
  </crew>
  <crew>
    <name>Atalanta</name> <role>Pitman</role> <role>Ballast</role>
  </crew>
  <crew>
    <name>Argos</name> <role>Shipwright</role> <role>Ballast</role>
  </crew>
  <need_repair>false</need_repair>
  <cargo_hold>5</cargo_hold>
  <cargo>goat</cargo>
  <cargo>golden fleece</cargo>
  <cargo>dragon's teeth</cargo>
</ship>

```

Schemas and Data Types

The fact that XML documents can carry DTDs is particularly useful in the context of service-oriented programming to specify (in a data-format technology-specific way) the data types expected by operations—both for validation and documentation. Indeed, XML has different so-called *schema* languages that, alongside DTD, can express constraints on the structure and content of XML documents—the main reason new schema languages evolved was to overcome the limitations of DTD specifications. Similarly, also JSON acquired standards, like JSON Schema, which can specify in a JSON-based format the structure of the expected (valid) JSON values.

For example, the piece of JSON Schema below is a possible validator for our illustrative ship JSON structure.

```

{ "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "name": { "type": "string" },
    "crew": {
      "type": "array",
      "items": {
        "type": "object",
        "properties": {
          "name": { "type": "string" },
          "role": { "oneOf": [
            { "type": "string" },
            { "type": "array", "items": { "type": "string" } }
          ] } } },
      "need_repair": { "type": "boolean" },
      "cargo_hold": { "type": "number" },
      "cargo": { "type": "array", "items": { "type": "string" } },
      "required": [ "name" ]
    }
  }
}

```

Without going into the details of each line, let us point out the interesting bits from the linguistic point of view.

First, we declare the version of the `$schema` specification we use to define our validator. This is not different from using a certain version of a given language implementation to define the types of our programs (where new versions can include new features; e.g., the inclusion of generics since Java 5.0, cf. Sect. 10.4.2). Schema technologies make the indication of this version explicit, so that any user of the schema knows what version of the specification it should be run against.

Then, we specify the type of the whole structure, which is an object (in the case of JSON, a string-key to value mapping). The `additionalProperties` key indicates whether or not to accept subtypes of the type we are defining; specifically, this is an interpretation of structural subtyping (cf. Sect. 8.5.2), where we can allow valid values to have more fields/properties than the ones defined in the schema. In this case, we forbid extensions of the structure specified in the schema (`additionalProperties` is set to `false`).

Next, we find the properties of valid values. For instance, we specify that the key `name` must correspond to a value of type `string`. The definition of the properties follows the usual nesting of type definitions, e.g., we indicate that the `crew` property is an array of objects, which we specify following the same convention that JSON schema provides for objects (made of properties, etc.). Notice that, in the definition of the objects of the `crew` array, we use the `oneOf` operator to define a sum type (cf. Sect. 8.4.3) for the values corresponding to the role key, which can either be strings or arrays of strings.

Lastly, we close our schema with the `required` key, which indicates which of the properties must be present to consider a JSON value valid. In this case, we set that valid values must have at least the outermost key `name` defined—which means that we accept any JSON value that has a valid `name` value and that adheres to any combination of the definitions of the other keys in the schema (applying a refinement notion of structural subtyping).

Jolie Types

Until here in this section, we discussed data types in close relation to the technology used to transmit the data among processes. For example, we can elect JSON as the serialisation technology used by our service and define the type of data exchanged in the form of a technology-specific definition: JSON schema.

However, while, we made a step towards interoperability (by fixing a common standard to exchange data), we have made a commitment against loose coupling, as we are forcing the clients of our service to emit and parse JSON values. What if other clients could only use some binary format or XML as serialisation technology? Allowing these to interact with our service entails two actions; a conceptual and a practical one. The conceptual one regards the definition of schemas (alternative to the JSON ones, if any) to validate the formatted data, and the extension of the interface presented by the service, which now should publish the availability of the new format, along with the pre-existing JSON-based definitions. The practical action is, of course,

to modify our service so that it can discern the desired formatting of the data and apply the specific procedures to translate it into/from memory.

As we have seen happening multiple times now (e.g., when we used parametric polymorphism to avoid the tedious and error-prone task of defining specific implementations of common, type-agnostic, data structures), abstraction can relieve programmers from the burden of the above routine—that one should endure every time the service must support a new data format.

In this case, we can have the language used to define the interface of our service abstract from the implementation details of the data-format technologies and provide a generic schema for type definition. The underlying guarantee to that abstraction is that the runtime of the language can either produce compliant schemas for the various technologies supported, allowing users to decide what solution best suits their needs, or dispense with those by implementing the same checks performed by the validators.

Both Alternatives Have Pros and Cons.

The second can provide a schema language even when the data format does not provide one and, as long as users can access the same data-format abstraction technology, they can use the abstract schema to produce and validate data serialised in any of the supported data formats. *Per contra*, when the second option is not applicable (e.g., because it is not available under a given technology stack), the generated, format-specific schemas would allow implementors to perform the validation regardless of the abstraction—this implies the availability of a schema specific to the target data format and it comes at the cost of re-generating and re-distributing the technology-specific schemas each time the types of the service change.

To make a concrete example, let us consider Jolie types. Jolie implements the second alternative presented above, i.e., it is a service-oriented programming language (first released in 2006) that provides type definitions which abstract from the data format used in data transmission.

Jolie types define structures as data trees, quite close to the ones we saw for the JSON and XML formats; in these trees, the root contains a value of some basic type (boolean, integer, string, etc.) and each node points to an array of trees. A possible Jolie data type for the JSON- and XML-based structures of the previous examples, which also follows the semantics of the JSON schema presented above, is the following:

```
type ship: {
  name: string
  crew*: {
    name?: string
    role*: string
  }
  need_repair?: bool
  cargo_hold?: int
  cargo*: string
}
```

The Jolie Programming Language

Jolie is an interpreted programming language designed to let developers program services in a simple, compositional way. In this chapter, we analyse the main elements of Jolie programs (types, interfaces, ports, services, and behaviours) and show how it provides a simple syntax for specifying the interfaces, the ports, and the behaviour of complex distributed programs. Besides these “fundamental” features, the language offers advanced constructs to support programmers in building complex distributed systems.

For instance, Jolie provides a set of architectural programming constructs that help users build complex compositions of services through a terse syntax. Some examples of these constructs are: “embedding”, which allows developers to run services within another, creating an efficient (e.g., communicating via internal memory) composite service (the construct is useful also because one can “detach” embedded services from their embedder by changing a couple of lines in their ports definitions, cf. Sect. 15.2.4); “redirection” and “aggregation”, which let programmers route messages among services in a structured way—the main difference between redirection and aggregation is that the former is known to the client, which needs to specifically state to which redirected service it wants to interact with, while aggregation is implicit and the client does not know whether it is interacting with an aggregator or not; “courier”, which enriches aggregation with a mechanism to inspect and modify messages at their reception (both before forwarding requests to their target services and before passing responses back to clients).

Another important language feature (specifically, of behaviours), is the provision of fault handling (succinctly exemplified in Sect. 15.2.5) with advanced features for ensuring that systems operate smoothly in the face of failures. In particular, Jolie handles faults through the use of scopes. Scopes are blocks of code that should execute together, i.e., a scope can mark a transactional section that shall either execute in its entirety or reverted otherwise. For this purpose, the language also provides a “termination and compensation” mechanism, where termination allows developers to specify what should happen if an operation fails, while compensation allows them to specify actions necessary to revert the effects of a failed operation.

The language has a strong industrial focus (e.g., its syntax recalls that of Java and C and it supports an ever-expanding array of data formats and communication protocols) and solid academic rooting—its initial formal foundations and implementation come from the University of Bologna and the project is currently led by the University of Southern Denmark, with the contribution of many international research and industrial organisations.

In the definition, we name our type `ship`. The root of `ship` is empty (`void`) and it contains `name`, `crew`, `need_repair`, `cargo_hold`, and `cargo` as nodes. In `ship`, `name` has the type `string`, and it is the only required node of the whole structure. This is evident from the `?` and `*` operators we find next to the other nodes, which indicate the cardinality of the elements in the arrays corresponding to each node—respectively, a zero-or-one and zero-or-more cardinality, while the absence of a cardinality specification indicates an exactly-one requirement (like that of the outermost `name`).

Interestingly, since nodes are arrays, we have a simpler definition than the one we saw using JSON schema. Indeed, with `role*: string` we express the fact that the `role` property of `crew` is optional and, if present, it can either contain a string (i.e., the unary array) or an array of strings—alternatively, one could capture the choice with Jolie sum types.

Another interesting point is noting the design choices on subtyping between JSON schema and Jolie. The former adopts an “open” interpretation of the type, where one needs to explicitly forbid the presence of `additionalProperties` and indicate what elements are `required`. The latter follows a “closed” interpretation of types, where defaults indicate exactly-one cardinality and subtyping is explicitly indicated, e.g., via sum types and nodes marked as `undefined` (not shown here). We invite the reader to ponder on the pros and cons of either approach as an exercise (e.g., from the point of view of a user that implements a service and/or a client).

As we see later in this chapter (in Sect. 15.2.4), the Jolie ports let the language offer data-format-abstract types to its users. Indeed, users define in ports the concrete format for receiving/transmitting data (e.g., JSON) and the Jolie runtime handles the transformation from/to the internal representation of the data. This also includes the validation of the data in reception/transmission, independently of the availability of a schema language for the selected data format.

Closing on data-format-abstract types, since the Jolie runtime handles both the validation and conversion of the data, at the moment there is no official support for type-to-schema synthesis, e.g., where one generates a JSON schema given a Jolie type—however, one can provide this support independently of the language runtime, e.g., with dedicated tools able to translate (compile) Jolie types into target data-format schemas.

15.2.3 Messaging Patterns

Since services are processes that communicate over some medium, besides agreeing on the formatting of data, they need to fix how they transmit invocations to each other. The general way to define how invocation and data transmission works between services spans three levels.

The lowest of these levels is that of *communication channels*, which identify the transmission medium (e.g., a wired or wireless connection) used to transmit the data. Examples, here, are the Ethernet and Bluetooth standards. Since the standards themselves have little to do with languages and, in particular, language-level abstractions, we do not further comment on them in our treatment.

Then, we find the *communication protocols*, which define how two or more parties can exchange information over a given communication channel. The communication protocol fixes (with reference specifications that frequently represent interactions in a way that reminds the above-all-parties abstraction of choreographies) the syntax of the exchanged messages, their sequence among the parties, and the rules that apply at the reception of each message, in response of the current state of the communication. For example, if during a communication one of the parties receives some corrupted data and the used communication protocol enforces error recovery, the protocol implementation shall spot the corrupted message and apply the rules put in place by its specification for that occasion—e.g., asking the sender to re-transmit the message. Representative communications protocols are the Transport Communication Protocol (TCP) and the User Datagram Protocol (UDP). We avoid commenting

on these for the same reason as communication channels, i.e., the protocols themselves do not have meaningful traits to comment on in our exposition on languages, but we mention them here because they will turn out—later in this chapter—to be characterising elements/parameters of service definitions.

The last level is that of *messaging patterns*, which concerns the logic of message exchange between the parties. While there exist many of these patterns, in the following, we focus on the main ones (and some of their variants).

Request-Response Pattern

As mentioned in the introductory part of this chapter, in a way, we already met the *request-response* (also called *request-reply*) pattern while presenting the stack-based control-transfer model (cf. Chap. 7). The pattern is one of the (if not the) most ubiquitous and, for instance, it describes the basic behaviour of the Web. Indeed, HTTP (a standard that defines how processes exchange messages in hypermedia information systems) is a request-response-based protocol that determines ways for Web browsers to interact with Web servers and manipulate resources (generally, Web pages). Since HTTP implements the request-response messaging pattern, all interactions between the parties in the protocol happen in pairs of requests and responses—specifically, the Web browser is the party that sends requests for some operations applied on some resources and the Web server processes those requests and sends back their responses. Notably, the pattern also dictates how errors work in HTTP, i.e., in all instances, the server shall respond to the request of the browser, which also includes the case of errors (via the famous 4XX and 5XX HTTP error codes, which the server shall return, unless some communication-level errors occurred to prevent the delivery of messages).

The request-response pattern is usually ascribed to the category of synchronous patterns, because its logic is generally interpreted as a synchronisation between the sender and the receiver, where the sender and the receiver both wait for each other's messages (where receivers and senders respectively wait for requests and responses). However, the pattern does not necessarily imply the synchronous consumption of both messages, but rather the fact that the communication between the two parties entails the transmission of two pairs of messages, one in one direction and the other in the opposite.

One-Way Pattern

Another pervasive messaging pattern is that of *one-way* communications, where the sender sends a message, without waiting for a response from the receiver. Also in this case, we can interpret one-way communications under different timings. The usual interpretation of one-way messaging is asynchronous, so that the sender transmits its message, ignoring the fact the receiver has consumed it—this does not exclude the possibility to wait for an acknowledgement of the correct reception of the message from the communication layer. The rarer, synchronous interpretation waits for the reception of an acknowledgement of the consumption of the message from the receiver.

Publish-Subscribe Pattern

The third main messaging pattern is the one called *publish-subscribe*. The distinctive trait of this pattern is that senders and receivers do not know each other's locations and cannot directly send or receive messages between themselves. Instead, the sender, called the *publisher*, of a message associates it with one or more classes of a given classification. The receivers of the message are those entities, called *subscribers*, that expressed interest in (i.e., subscribed to) one of the classes associated with that message. While implementations vary and can realise point-to-point semantics for message delivery (where only one of many subscribers actually receives the published message), the most frequent interpretation of the pattern follows a multicast semantics, where all subscribers to a class can receive a published message associated with that class. Advanced implementations let users define (e.g., per class) further refinements of the publish-subscribe semantics, e.g., with delivery policies. Simplifying, the most common ones are: *at-most once*, where the message is sent to subscribers only once, without any guarantee of actual reception; *at-least once*, where subscribers can receive one or more copies of the same message, until they confirm the reception of at least one copy; *exactly once*, where the subscriber receives only one copy of the message, thanks to a confirmation protocol that checks the presence of the subscriber, delivers the message, and either receives the acknowledgement of its consumption or repeats the previous actions, until the reception of the acknowledgement.

In implementations, publish-subscribe messaging frequently entails the presence of a third entity (next to publishers and subscribers) in the form of either a *message broker* or a *bus*. The role of this entity is to handle subscriptions and the routing and delivery of the published messages, where more advanced brokers can apply complex logic for the manipulation and routing of messages. For example, some brokers (e.g., Apache Kafka) allow subscribers to define their interest classes via some logical formula applied to each message (so, the subscription would not be to nominal classes, but rather define some conditions that classify a message as “interesting” for that subscriber). The *Enterprise Service Bus* (ESB), found in first-generation service-oriented architectures (cf. Sect. 15.3.1), is an advanced kind of bus that provides features like data transformation and protocol conversion.

15.2.4 Operations and Interfaces

From the point of view of languages, we need to let services express, in their interfaces, what messaging patterns they expect their operations to follow. As seen for data types, how services communicate this information to their clients vary, but we can roughly partition these modalities into two groups: explicit and implicit.

Explicit Messaging Patterns

To let services explicitly define the messaging pattern that characterises each of their operations, we need some language for that purpose.

An early exponent of this class of languages for service orientation is the Web Services Description Language (WSDL); a language dating back to the year 2000 and able to define the data types (mainly as XML-typed documents), the interfaces, and the contact points provided by the service (we analyse the latter in Sect. 15.2.4). Since WSDL is a verbose XML-based format, we limit our example to meaningful snippets of messaging patterns and interfaces and avoid showing the type definitions (which follow the discipline overviewed in Sect. 15.2.2).

```
<interface name="PortInterface">
  <operation name="lookupShip" pattern="http://www.w3.org/ns/wsd1/in-out">
    <input messageLabel="In" element="tns:string"/>
    <output messageLabel="Out" element="tns:ship"/>
    <fault name="ShipNotDocked" element="tns:string"/>
  </operation>
  <operation name="dock" pattern="http://www.w3.org/ns/wsd1/in-only">
    <input messageLabel="In" element="tns:ship"/>
  </operation>
</interface>
```

In the snippet above, we find the definition of a `PortInterface` with two operations, called `lookupShip` and `dock`. The `lookupShip` operation follows a request-response pattern (`in-out`, in WSDL parlance) where the sender sends in its request the name of a ship (as a string) and the service responds with a ship data structure (e.g., the XML structure illustrated in Sect. 15.2.2). The `dock` operation is a one-way one (`in-only`) and accepts a ship data structure—the operation simulates the docking of a ship into a harbour. Since the `lookupShip` operation simulates the fetching of the information of docked ships, we need to communicate to the user the possible failure of the lookup, e.g., occurring when no ship with the name requested by the client ever docked into the harbour. We signal this possibility in the WSDL interface by associating the `ShipNotDocked` fault to the `lookupShip` operation. For completeness, besides “`in-out`” and “`in-only`”, WSDL provides also a consumption-acknowledged one-way, called “`robust in-only`”, and a request-response with an optional response, named “`in-optional-out`”. The standard also defines complementaries of the aforementioned patterns (for a total of 8), which one can use to describe the complementary interface of the other party in the interaction.

As an additional (and possibly easier-to-read) example, let us look at an alternative version of the above interface, written in Jolie.

```
interface PortInterface {
  requestResponse: lookupShip( string )( ship ) throws ShipNotDocked( string )
  oneWay: dock( ship )
}
```

In the previous snippet, we use the Jolie interface keyword to define our `PortInterface`, which contains the two operations seen in the WSDL example: `lookupShip`, which is a `requestResponse` that can throw a `ShipNotDocked` fault, and the `oneWay` `dock`. At the moment, `requestResponses` and `oneWays` are the only messaging patterns supported by the language, totalling to four when considering their complementaries, called respectively “`notification`” and “`solicit-`

response”, that the language infers from the association of interfaces and ports, discussed in Sect. 15.2.4.

Implicit Messaging Patterns

The implicit way of specifying the messaging patterns of an interface (and the operations therein) is an approach similar to defining the data types of (the operations of) a service in a specific data format: we pick a technology that fixes the messaging pattern of the operations it supports and we use it to define the interface of the service.

A ubiquitous example of this kind of definitions is the Representational State Transfer (REST) architectural style. The original proposal for REST (by Roy Fielding, who worked on the HTTP1.1 standard at the turn of the millennium) came as a way to describe a given service interface—also called its Application Programming Interface (API)—using Uniform Resource Locations (URLs), the HTTP methods (GET, POST, PUT, etc.), and an HTTP-supported data format (e.g., URL encoding, JSON, XML). The intent of the proposal was to provide a way for developers to take advantage of the existing Internet infrastructure (e.g., caching idempotent operations, transparent proxying) when developing services aligned with the use case of HTTP (e.g., for hypermedia data transfers that obey the same constraints of HTTP). Notwithstanding its proponent’s intentions, REST has been generally adopted for the specification of service interfaces, mainly as a much simpler replacement to the complexity of SOAP—Simple Object Access Protocol; a (ironically, given its name) sophisticated messaging protocol based on XML and HTTP that encompasses data types and interface definitions, message delivery, and the peers’ topology (the protocol later inspired the definition of WSDL, dedicated only to interface-and-access specification). By defining a service (interface) as “RESTful”—so called because it follows the constraints dictated by the REST style—the developer not only implicitly makes a set of technology commitments (e.g., on the usage of HTTP) but they also declare that the operations correspond to the HTTP methods and, by definition of the standard, that they all follow a request-response pattern.

Similar to format-specific and format-abstract data type definitions, messaging-pattern definitions show an abstraction property between explicit and implicit ones. For example, given an abstract WSDL or Jolie interface, we can specify a service implementation that offers a RESTful version of the abstract one by using the HTTP stack as the communication protocol and, if needed, by mapping operations to HTTP methods and resources (as we do in Sect. 15.2.4). While adapting a RESTful interface to other technologies is possible (e.g., via a middle service that translates requests from one format to the other, similar to what ESBs do as part of their protocol mediation functionalities), the style does not provide messaging-pattern information and has no standard way to express other patterns than the request-response one.

Even though REST increments the coupling among services (which need to all “speak” REST to interact with each other) its simplicity largely won over more complex alternatives, and, at the time of writing, it is one of the major styles to define a service interface—so much so that, e.g., the Constrained Application Protocol (CoAP), a widely-adopted communication protocol for low-power, resource-constrained devices, follows the HTTP model with the declared purpose of offering direct support (i.e., coupling) to and integration with RESTful services.

Bindings and Ports

By now, the reader might have acquired the sense that, when specifying service interfaces, we have two main options.

The first, more concrete, is to directly describe interfaces in terms of the actual technologies used to implement the communication (e.g., by choosing the JSON data format, possibly define types with the JSON Schema, and describe the operations in a RESTful style).

The second alternative, more abstract (and loosely coupled), is to use language constructs that let us define the interfaces independently of the actual technologies that characterise a service implementation (e.g., using Jolie types and interfaces). Like any abstraction, we need to complement it with constructs that let us express how to ground them, i.e., to specify what technologies we want to use to implement the interfaces. These constructs correspond to (a) a *binding* or mapping of interfaces to a concrete combination of data format and communication technologies and (b) the details of a specific *port* for the binding, which practically determines how users can reach the service, e.g., the network address of the service, the information to connect to a bus/message broker, etc.

In WSDL, the designers chose to allow for an even more fine-grained separation between interfaces, bindings, and ports (called *endpoints* in recent versions of the language), through the introduction of the optional construct of *port types*, which collect sets of interfaces under the same port name. However, to describe a WSDL service, it is sufficient to (i) define a binding of a port to a pair of concrete data format and communication technology; (ii) specify a port that completes the binding with technology-specific information to expose the service to users. Thus, a WSDL *service* results from a collection of ports, which expose some collection of interfaces over a possibly heterogeneous set of endpoints (using different data format and communication technologies).

For instance, we can complete our “Harbour” example with a WSDL `<service>` that exposes the `PortInterface` (cf. Sect. 15.2.4) as a RESTful service.

```
<binding name="HttpBinding" interface="tns:PortInterface"
  type="http://www.w3.org/ns/wsdl/http">
  <operation ref="tns:dock" http:method="POST">
    <input> <mime:content type="application/xml" </input>
  </operation>
  <operation ref="tns:lookupShip" http:method="GET">
    <input> <mime:content type="application/xml" </input>
    <output> <mime:content type="application/xml" </output>
  </operation>
</binding>

<service name="Harbour" interface="tns:PortInterface">
<port name="RestEndpoint"
  binding="tns:HttpBinding"
  address="http://www.iolcus.com/harbour"/>
</service>
```

In the snippet above, note that the HTTP binding does more than just specifying the technology used to realise the interface. Indeed, the binding also maps the operations to HTTP resources, available through specific HTTP methods, and it indicates the format of the exchanged data (xml). Specifically, since `dock` receives some new information, we map it to a POST (the method indicates both the submission of some data to the resource and a possible change in the state of the recipient), while we map `lookupShip` to a GET (a request for data retrieval).

The paradigm of bindings and ports for abstract interfaces carries also to Jolie, although with a couple of different design choices with respect to WSDL. First (as noted in Sect. 15.2.4), WSDL declares in its interfaces both the messaging pattern (e.g., one-way) and the direction of the communication (e.g., “in-only” or “out-only”, depending on whether the service shall respectively receive or send the one-way message). In Jolie, interfaces only indicate the messaging pattern, while it is the kind of port the interfaces are bound to that determines the direction of the communication. For example, we can define in Jolie an equivalent endpoint to the one above in the following way

```
inputPort HTTPport {
  location: "socket://www.iolcus.com:80/harbour"
  protocol: http {
    osc.dock.method      = "post"
    osc.lookupShip.method = "get"
    format = "xml"
  }
  interfaces: PortInterface
}
```

where, by defining an `inputPort`, we indicate that the service exposes the `PortInterface` as inbound operations. Indeed, Jolie provides two types of ports: `inputPorts` and `outputPorts`, respectively indicating that the operations found in the collection of `interfaces` (where one can specify a comma-separated list of interface names) associated with that port are inbound or outbound.

The second difference with WSDL is that Jolie ports integrate the concept of bindings by including the definition of a `location` and a `protocol`. The `location` indicates both the medium and transport protocol used by the port. In the example above, the prefix `socket://` specifies the usage of a TCP socket over the Internet Protocol (IP) that accepts connections at the address `www.iolcus.com/harbour` on the traditional HTTP-server port 80—WSDL partially omits this element by assuming a TCP/IP stack for HTTP bindings. The `protocol` of a port defines the communication and data-format technologies for message exchange. In the example, we indicate `http` as the communication protocol, specifying (like in WSDL) the HTTP methods that the operations of the interface map to and that the format of the exchanged data is `xml`. The orthogonal take of Jolie with ports and interfaces makes it easy to change how a service exposes its operations. For example, we could change the `location` to `"bt12cap://..."` and let the service communicate via the bluetooth medium. Similarly, we can modify the `protocol` parameter to support other ways in which the service expects to receive and responds to requests. For instance, we can switch from the generic `http` pro-

toocol to the more specific `jsonrpc` or `xmlrpc` ones (which use HTTP but define standards for operations and data in either data format) or, to save bandwidth, the `sodep` binary format. In all these cases, these changes do not affect interfaces (and, as we are going to see, behaviours), which do not vary.

15.2.5 Behaviour

Orchestration

As presented in the introductory sections of this chapter, orchestration is the most established and used paradigm for behaviour definition in service-oriented programming. We can attribute its appeal to two facts. First, the approach is a natural extension to distributed systems of sequential programming (i.e., the one we presented and discussed until Chap. 14, on concurrency). Second, as we see in the next section, the complementary approach, on choreography, has yet to find an established interpretation, which is still an active research topic.

Orchestration lets programmers design and reason on the logic of (concurrent) distributed programs in a simple way, which might also help to avoid deadlocks (cf. Sect. 14.5.3). The orchestrator confines in one place the coordination logic of the distributed system, while the other participants are reactive components that can always respond to invocations to any of their offered operations. Drawing a parallel, we can compare the orchestration paradigm to sequential programming, where the orchestrator represents the “main” execution block of the program that calls the operations of the other modules, necessary to complete the block’s task. This also means that, using orchestration languages, programmers do not need to learn from scratch a paradigm for service orientation, and they can rather build upon the knowledge and experience they gained with other paradigms.

We can examine the above observations on orchestration from a practical point of view: as shown in the next sections, we do not necessarily need a service-oriented language to define the behaviour of an orchestrated service-oriented program. For example, we see how one can define some clever mappings to close the abstraction gap between object- and service-orchestration and use an object-oriented language (e.g., Java) to program an orchestration service.

However, building services using a language not built upon the service-oriented paradigm brings complications, such as making it difficult to identify and reason on the conceptual elements that characterise a service. Indeed, what we do is mapping elements of the (non-service-oriented) implementation language to elements of the service-oriented program, e.g., we map some objects/classes to data/types, other objects to services and some of their methods to operations (with their specific messaging patterns) and endpoints. Essentially, we are fragmenting the interface of the service over many components, making it difficult for programmers to gain an overview of the actual interface that the service ends up exposing (this is sometimes relieved with tools that can statically analyse the codebase and assemble the programmed interface using one of the styles and/or languages seen in the previous sections). Moreover, programming in a different paradigm than service orientation

can lead developers to also scatter the logic of orchestration over multiple non-service components, making it difficult to follow its steps or reuse the implementation in other codebases (e.g., due to the common problem of object-oriented languages where e.g., a class comes with dependencies that the user needs to also import if they want to use it²).

These limitations (already apparent with the introduction of remote procedure calls) motivated the introduction of service-oriented languages for the programming of service behaviours, where all interacting entities are services.

In the remainder of the section, we first provide an illustrative (snippet of) a service that implements our “Harbour” interface and of an orchestrator that calls the operations of the Harbour service. To do this, we use Java and a dedicated framework that exploits annotations to conventionally configure the many technical details of interface specification and instantiation. Then, we briefly introduce the Business Process Execution Language (BPEL) and Jolie behaviours as two example service-oriented orchestration languages. We highlight the distinctive service-oriented features of this class of languages, and we illustrate their usage by translating our Java services into Jolie ones.

Behaviour in Java

To exemplify how one can use an object-oriented language to implement services in orchestration, we consider Java and the (at the time of writing) widely popular Spring framework, first introduced in 2003.

A Simple Orchestrator

Let us begin by looking at a possible implementation of the Harbour service, reported below.

```
@RestController
public class Harbour {
    @PostMapping("/dock")
    void dock( @RequestBody Ship ship ) {
        ships.add( ship );
    }
    @GetMapping("/lookupShip/{name}")
    Ship lookupShip( @PathVariable String name )
        throws ShipNotDocked {
        return ships.findByName( name );
    }
}
```

An interesting point, from a linguistic perspective, is the usage of Java annotations—the @-notation found in the code above, which annotate the symbol found on their right—for the mapping. In the snippet, we declare that the Harbour

² Joe Armstrong, the principal inventor of the Erlang language, was quoted as quipping about object-oriented modularity: “You wanted a banana but what you got was a gorilla holding the banana and the entire jungle”.

class is a `@RestController`. To the Spring runtime, this means that it shall consider the (annotated) methods of this class as operations of a RESTful service, available at some TCP/IP location (omitted here). Moreover, the annotation also entails the (possible) automatic (de)serialisation between some configured data format (here, JSON, by default) and Java objects. In the code, we map the `dock` object method to the HTTP POST (`@PostMapping`) method invoked on the `/dock` HTTP resource. The `@RequestBody` annotation indicates that the invocation should carry a value, which should comply with the definition of the `Ship` object (also omitted here—we leave the possible translation as Java classes of the `ship` data structure illustrated in Sect. 15.2.2 as an exercise to the reader). The Spring runtime takes care of deserialising the JSON value of the request into a `Ship` object, expected by the `dock` method. In the body of the method, we find the effect of the invocation: we add to the `ships` object (a repository-like data structure) the `ship` we obtained from the invocation. The definition of the `lookupShip` operation follows a similar logic. In this case, we indicate the usage of the GET (`@GetMapping`) HTTP method, and we map the invocation on a resource address parametric to the request value, i.e., we prefix any invocation to the `lookupShip` operation with the `/lookupShip` fragment and let the request variable name be the parameter of the address (as indicated by the `@PathVariable` annotation). Since we return a `Ship` object, the Spring runtime also performs the opposite work it did for the `dock` operation, i.e., it also serialises a Java object into a JSON one, sent back as the response. Note that, although not explicit from the syntax of the annotation, the framework translates any value a method can “output”; in this case, the values correspond to either (i.e., the sum type of) the declared return type (`Ship`) and the “fault” `ShipNotDocked`—more precisely, the values would map respectively to a success and an error HTTP response.

Considering loose coupling and interoperability, this solution (although fragmenting the service elements over different modules) satisfies both principles. Annotations represent mappings that allow the Java code to abstract from the implementation of the caller/callee (loose coupling) and one can change the technologies used by the service without necessarily modifying the behaviour (interoperability).

However, the service we exemplified above is the simplest form of orchestrator, i.e., one that does not need to coordinate other services to implement the distributed logic of the system. Unfortunately, looking at a more complex scenario reveals that the support for interoperability and loose coupling given by this class of solutions becomes less satisfactory.

Orchestrating Services

Let us consider a richer scenario, where we have several Harbour services, i.e., possibly different implementations of the same Harbour service interface, and a “Port Authority” orchestrator that interacts with these to coordinate inter-port operations. For simplicity, we fix the number of Harbour services to three and consider only one search request-response operation implemented by the Port Authority service. With `search`, the service receives the name of a ship (a string), and it interacts with

other services to see whether a ship with that name ever docked at one of the Harbours. In case all Harbours miss that entry, the service responds with a ship-not-found fault.

Notice that the behaviour we just described does not specify how the interactions with the Harbour services happen. Are those sequential and, in case, following what order? Are they performed in a *fork-join* fashion?³ For this Java example, we limit ourselves to looking at the simpler sequential version, as the parallel one would involve the more complex programming of threads (cf. Sect. 14.7).

```
@RestController
public class PortAuthority {
    Optional< Ship > lookupAtHarbour( String address, String name ) {
        return WebClient.create( address )
            .get()
            .uri( "/lookupShip/" + name )
            .retrieve()
            .bodyToMono( Ship.class )
            .onErrorResume( e -> Mono.empty() )
            .blockOptional();
    }

    @GetMapping("/search/{name}")
    Ship search( @PathVariable String name ) throws ShipNotDocked {
        return lookupAtHarbour( "http://harbour1.com", name )
            .orElse( lookupAtHarbour( "http://harbour2.com", name )
                .orElse( lookupAtHarbour( "http://harbour3.com", name )
                    .orElseThrow( ShipNotDocked::new ) ) );
    }
}
```

Above, we use the Java `Optional < T >` type—a parametric type that encapsulates a possible value and provides operations to check its presence and to obtain it—to express in a succinct way the logic of the orchestration, contained within the `search` method. In the method, first, we perform a lookup at the first Harbour service (with address `http://harbour1.com`). To do this, we call the `lookupAtHarbour` method, which returns an `Optional < Ship >`. The `Optional < T >` type provides the `orElse` method that either returns the value wrapped in the `Optional`, if present, or it allows us to run some code. Here, we exploit the `orElse` method to call the second and third Harbour services in sequence (with respective addresses `http://harbour2.com` and `http://harbour3.com`). If none of the invocations provided a value, we throw a `ShipNotDocked` to the caller (the framework translates it into an HTTP error) with the `orElseThrow` method—which throws an exception in case of an empty `Optional`.

The `lookupAtHarbour` method uses the `WebClient` provided by the Spring framework to implement the call to the REST interface of the Harbour(s) (the `lookupShip` operation, in particular), retrieve the response, and convert it into the `Optional` value illustrated above.

The main takeaway from the example above is that the discontinuity and limitations of the mapping to services become more evident as the complexity of the

³ In a way that each invocation executes in a parallel branch and they eventually join the main branch to resume sequential execution.

logic of the interaction grows. In particular, while in the Harbour implementation the Java code remained relatively independent of the annotations that bridged between services and the objects, here, we have the “service code” (e.g., the one needed to perform the connection to a service or the invocation of an operation) leak into the code of the Java classes. This kind of implementation marks a strong commitment that increases the coupling between the behaviour of the orchestrator and the communication technologies that the invoked services adopt. This approach also undermines interoperability, since changing or including different communication technology stacks likely requires the rewriting of large portions of the orchestrator.

BPEL and Jolie Behaviours

The name “BPEL” does not exactly identify a language, and it rather indicates a family of close dialects (e.g., BPEL4WS, OASIS WS-BPEL, Oracle BPEL) united by the same XML-based syntax and constructs, whilst not strictly adhering to the same semantics (i.e., the same BPEL program executed on different BPEL runtimes can have slightly dissimilar behaviours).

The relevant trait of the BPEL (family of) language(s) is that it describes programs in terms of inbound receptions and orchestrations (called business processes) of external entities. Hence, the language is service-oriented in the sense that a BPEL program describes the behaviour of a service (an orchestrator) and every other entity the program can interact with is a service as well. More specifically, BPEL uses WSDL descriptors to obtain the definition of both the operations it needs to implement (the inbound requests) and the interfaces and ports of the services it shall orchestrate to complete some task.

Jolie behaviours follow a similar take. They are compositions of inbound actions and of invocations to external services, defined by the `inputPorts` and the `outputPorts` of the service (which, in turn, specify the interfaces expected and offered by the service).

To make things more concrete, let us see a possible transposition of the Java code we wrote earlier as Jolie services.

A Simple Orchestrator

```

service Harbour {
  inputPort HTTPport { /* ... */ }
  outputPort Ships { /* ... */ }

  execution { concurrent }

  main {
    [ dock( ship ) ]{
      add@Ships( ship )
    }
    [ lookupShip( name )( ship ){
      install( default => throw( ShipNotDocked, name ) )
      findByName@Ships( name )( ship )
    } ]
  }
}

```

Notice that, in the code above, we use the `HTTPport` defined in Sect. 15.2.4. Then, we choose to orchestrate a sub-service, bound by the outputPort `Ships`, which implements an interface (omitted here) similar to the one we saw for the `ships` objects in our Java example. The important difference with our Java example is that, here, there is no discontinuity determined by the passage between services and objects. Indeed, `Ships` is itself a service which our Harbour service orchestrates to store and retrieve information on ships.⁴

Above, the `main { ... }` block indicates the entry point of execution of the behaviour of the service (like the traditional `main` procedure/method found in C or Java programs). Within the `main` block, we find the `dock` and `lookupShip` operations composed using a guarded choice.

To explain how guarded choices work, let us assume three one-way operations (for simplicity) `a`, `b`, and `c`. Then, the code

```
[ a() ] { /* Ba */ } [ b() ] { /* Bb */ } [ c() ] { /* Bc */ }
```

means “wait to either receive an invocation on `a`, `b` or `c`, execute the continuation within the associated block (`Ba` for `a`, `Bb` for `b` or `Bc` for `c`), and discard the other branches (e.g., if `a` is invoked, we would discard the `b` and `c` branches)”. Note that in our example, the `lookupShip` branch misses the trailing `{ ... }` block. This is a contracted form of the guarded-choice operator for empty continuations—mind to not confuse the block within the square brackets (explained later) for the continuation.

Before returning to describe the behaviour found in the body of the `main` block, notice the `execution {concurrent}` instruction between the ports and the `main` block. This instruction specifies how the Jolie runtime shall handle the instantiation of this service. Since services can hold state, service instantiation is similar to object instantiation: we want to specify if/when to create a new instance to manage distinct pools of data. In object orientation, we traditionally find constructs such as `new`, called from some other object’s context in the same runtime. Differently, in service orientation the requester of the instantiation is necessarily a component that is outside the service runtime and, thus, we need to indicate which operations are available to start a new instance of a service. In our Java example, this was implicit in the creation of the `RestController` object. Jolie makes the logic of service instantiation explicit through the interaction between the `execution` instruction and the behaviour. The former indicates how the runtime shall create new instances of the service, following three modalities: `single`, `sequential`, and `concurrent`. The first creates a single instance of the service which, once it reaches its last instruction, terminates. The second and third modalities can create an unbound number of instances of the same service, with the difference that the second waits for the preceding instance to finish before starting, while the third does not. Since the latter

⁴ One possibility is to implement the `Ships` service in Jolie, using in its behaviour the basic data manipulation primitives provided by the language. Alternatively, Jolie supports delegating the implementation of operations to other languages (e.g., Java, JavaScript) through the `embed` command. This essentially allows us to map service operations to methods/functions without mixing the two paradigms in the same code artefact.

two modalities can start a new instance of a service from an invocation of a client,⁵ they interact with the behaviour to define which of the operations in the service's interface can start a new instance. Specifically, all input operations at the top of the `main` block can instantiate a new copy of the service, while the others are accessible within a running instance.⁶

In our example, both the `dock` and `lookupShip` operations are top-level inputs and, thus, either invocation can start a new instance of the service. Since we declare the execution of the service `concurrent`, all instances run in parallel.

In the two branches, the `dock` operation orchestrates the invocation of the `add` operation at the `Ships` service. The code for service invocation uses the syntax `o@S`, where `o` is the operation label and `S` is the name of the `outputPort` binding the invoked service. The syntax for `requestResponses` and `oneWays` seen for interfaces carries to Jolie behaviours both in inputs and outputs (the latter, adding the `@S` binding). We briefly focus on the particular case of inbound `requestResponses`, like `lookupShip`, where the first pair of parentheses encloses the variable holding the content of the request (e.g., `name`) and the second pair wraps the variable (`ship`) that carries the response to the invoker. At runtime, inbound request-responses follow the steps: receive the invocation, execute until the completion of the code within the associated block `{ . . . }`, evaluate the response. Hence, in our code, the invocation of the `lookupShip` operation stores in the `name` variable the value of the request. Then, it invokes the `Ships` service to retrieve the data, which stores its response in the `ships` variable. Since this is the last instruction in the block, the `lookupShip` operation sends back its response, which is the content of the `ships` variable. The `install` instruction found within the same block indicates the instantiation of a fault handler. In this case, we use the generic `default` label to target all possible errors generated by the `findByName` operation and report (`throw`) the problem back to the invoker as a `ShipNotDocked` fault (in place of a valid `ship` value, as per `PortInterface`).

Orchestrating Services

Although we already saw service orchestration in the previous example, let us complete our comparison with the previous Java example by presenting an alternative version of the Port Authority in Jolie.

In particular, we have the Port Authority invoke the three Harbours of the example in parallel. This allows us to see other two Jolie behaviour composition operators (besides the guarded-choice commented above): the parallel `|` (reminder of the parallel composition operator seen in Sect. 14.6.1) and the sequence `;` one (the latter

⁵ Also the `single` modality starts a (single) new instance, but, since the instantiation happens at the start of the service, there is no need to wait for an invocation from a client to do it.

⁶ The service-oriented way of handling request-to-instance correlation is via (data) *correlation sets*, i.e., a technique whereby request messages contain specific parts that the invoked service can use to find the instance that message is intended for. While also this matter would deserve a “language” treatment (on how it can impact types, interfaces, and behaviours), we avoid to cover also this part and keep our presentation focussed on the fundamental elements of the paradigm.

is the default one when the programmer omits to specify a composition operator and just separates instructions with a new line).

```

service PortAuthority {
  outputPort Harbour1 { /* ... */ }
  outputPort Harbour2 { /* ... */ }
  outputPort Harbour3 { /* ... */ }
  inputPort in { /* ... */ }

  define faultHandler {
    install(default=>skip)
  }

  main {
    search( name )( ship ){
      {
        scope(h1){faultHandler; lookupShip@Harbour1(name)(ship)}
      | scope(h2){faultHandler; lookupShip@Harbour2(name)(ship)}
      | scope(h3){faultHandler; lookupShip@Harbour3(name)(ship)}
      }
      if( ! is_defined( ship ) )
        throw( ShipNotDocked, name )
    }
  }
}

```

Starting from the bindings, the Authority Port service has three outputPorts, one for each Harbour it interacts with, and an inputPort (in) to receive search requests from its clients.

At invocation, the orchestrator calls in parallel the three Harbours. This is visible from the composition `... | ... | ...` at the beginning of the search request-response block. We use blocks for each invocation because we want first to install a separate fault handler for each lookup request and then perform the related request. The three blocks execute in parallel, but the instructions therein follow the order: first, install the handler and then, perform the invocation. Since we handle faults in the same way for any invocation, we define a procedure, named `faultHandler`, which contains the fault-handling logic—we associate each (`default`) handler with a procedure, named `skip` (omitted), that discards the fault—and we call `faultHandler` from inside the blocks. We prefix each block with a `scope`, e.g., `scope(h1){...}`, to specify that we manage the fault thrown by a Harbour within its containing block. Besides fault handling, we use blocks to explicitly indicate the precedence among the composition operators. Another example of this way of specifying the composition of instructions is the outer block that wraps the three inner ones put in parallel. In this case, we are indicating that the three blocks shall all fully run, after which we can proceed with the code following the outer block (which also serves as an example of a *fork-join* composition).

Before moving to the last lines, we notice a detail; Jolie variables have module scope. While in block-scoped languages (e.g., C, Java) variables are valid within the block where they are declared, in module-scoped languages the scope of a name binding is the module (in this case, the service) that contains them. This means, e.g., that the `ships` found in the three `lookupShip` invocations in the example refer

to the same variable, which each might overwrite (more on this at the end of the section).

Completing the example, we check that at least one of the Harbours returned a valid `ship` value (i.e., the invocations assigned to it a value) using the `is_defined` predicate. If false, we throw a `ShipNotDocked` fault.

As a final remark, we invite the reader to ponder on the two ways we composed the invocations of the Harbour services and what that entails from the point of view of the distributed logic of the composition.

In the Java example, we followed a sequential, conditional composition. If the first Harbour returned a valid response, we would avoid querying the other Harbours. Thus, we would consider the first “good” response as valid and skip any further invocation. This imposes a ranking among the Harbours. Even if the second Harbour had an entry for the searched ship, if the first Harbour responded with a valid value, our orchestrator would consider it the only valid one (and the same goes between the second and third Harbours, when the first misses an entry for the searched ship and both of the former have a valid one).

Instead, in the Jolie example, we query the three Harbours in parallel and store their responses within the same `ship` variable. What could happen at runtime is that one of the Harbours responds first and sets the content of the `ship` variable, followed by the response of a second one, which overwrites the content of `ship` set by the first one.⁷ The behaviour we specify is that the last Harbour to respond with a valid value is also the one that last sets the `ship` variable, which is then used to respond back to the user. Hence, in this case, it is not the programmer who defined the precedence among the Harbours, instead determined by the response times of the Harbours and the non-determinism of the parallel invocations.

In general, more advanced approaches exist, usually referred to as composition or workflow patterns, which constitute a suite to compare the expressiveness and ergonomics of different orchestration languages—in our case, alternatives are invoking all Harbours in parallel, waiting for the first valid response, and discarding the others or querying all the Harbours (in sequence or parallel) and successfully responding only if they all have compatible entries of the same ship.

Choreographies

As presented in the introductory sections of this chapter, in service-oriented programming, we tend to cast orchestration and choreographies in a dual relationship. While this can hold from a conceptual point of view—contrasting the central-point-of-control approach of orchestration against the distributed-coordination-logic of choreographies—it is more difficult to draw clear-cut parallels between the existing interpretations of the choreographic approach and orchestration.

Indeed, orchestration has a consolidated understanding, both among theoreticians and practitioners, as a style of programming distributed (service-oriented) systems.

⁷ The Jolie runtime treats variable assignments atomically, so these are always linearised (in a non-deterministic way) and it cannot happen that, e.g., the `ship` variable contains a “mix” of two responses received in parallel.

Contrarily, it is more difficult to find an agreement on what could be the “mainstream” interpretation of choreographies. We can trace at least three main lines of work (which can also intersect) that found their roots in different interpretations of what a choreography is and from which derive different tools that programmers can use in the development of choreographic service-oriented programs. These interpretations see choreographies as either a specification, a (behavioural) type, or a program. The latter is the closest interpretation we can frame as dual to orchestration, but any of these has yet to gain wide adoption in service-oriented programming.

Choreographies as Specifications

The earliest interpretation of choreographies in the context of service orientation is that of understanding and using these tools for designing and specifying the behaviour of a distributed program.

This category includes also (typically diagrammatic) languages that predate service orientation, like the mentioned Alice-and-Bob notation as well as sequence diagrams. Since these tools are ubiquitously used in the design and specification of protocols and distributed components, we could say that the “specification” is the most common interpretation of choreographies. However, here, we focus on the languages purposefully introduced for service orientation.

The most famous service-oriented language in this category is the Web Service Choreography Description Language (WS-CDL). Quoting its W3C Candidate Recommendation (CR), “[WS-CDL] is an XML-based language that describes peer-to-peer collaborations of participants by defining, from a global viewpoint, their common and complementary observable behaviour; where ordered message exchanges result in accomplishing a common business goal”. The CR further specifies that WS-CDL is not an “executable business process description language” or an implementation language, leaving the task of specifying the execution logic of applications to other languages (e.g., BPEL, Java, etc.).

The main elements of a WS-CDL choreography are:

- the types of the data exchanged in the distributed program;
- the participants that exchange the data, which impersonate a certain *role* in the choreography;
- the channels that connect the participants, specifying where and how to exchange the data;
- the interactions (or data exchanges) that can occur in the distributed program.

Like BPEL, also WS-CDL integrates with other WS-* technologies, above all WSDL, which can define types, operations, and bindings, to a given WS-CDL choreography. Here, since the language is quite verbose, we illustrate the shape of a WS-CDL choreography using a pseudo-language inspired by the latter.

```
choreography HarbourChoreography {
  /* ... */
  type ship { /* ... */ }
  type ShipNotDocked extends Fault
```

```

roles: Client, Harbour
channel chan {
  oneWay: dock(ship)
  requestResponse: lookupShip(string)(ship) throws ShipNotDocked
}
main {
  /* ... */
  if ( Client.action == "dock" ){
    chan.dock: Client.ship -> Harbour.ship
  } else {
    chan.lookupShip: Client.name -> Harbour.name
    try {
      Harbour.ship = Harbour.findByName( name )
      chan.lookupShip_: Harbour.ship -> Client.ship
    } catch ( ShipNotDocked ){
      chan.lookupShip_: Harbour.ShipNotDocked -> Client
    }
  }
}
}

```

In the example, the main entity of our language is the choreography. The fact that we give it a name, *HabourChoreography*, signals that choreographies are the modules of our language, which we can assemble, import, and use within other modules (we leave out choreography modularity in our simple examples).

For brevity, we borrow Jolie data types for our pseudo-language (e.g., the *ship* type saw in Sect. 15.2.2) and use subtyping for Java-like fault handling (e.g., *ShipNotDocked* subtype of *Fault*). Then, we list the roles in our choreography—a *Client* and the *Harbour*—and we specify the channels we are going to use in the choreography. The attentive readers might have already drawn the parallel between channels and interfaces. Indeed, channels specify the kind (e.g., *oneWay*, *requestResponse*, also borrowed from Jolie for this example) of operations and the involved data types of the possible interactions among the roles.

Finally, we have the specification of the global-viewpoint behaviour among the roles. In the example, we define a *main* block where we see a branching behaviour, represented by the *if* on the *action* variable of the *Client*. This means that it is the *Client* who decides what to do next, communicating that decision to the other participants (here, only the *Harbour*).

While conditionals and branching are traditional elements of local programming, these assume a more nuanced connotation in choreographies, known as the *knowledge of choice*. Indeed, in a local program, the branching regards only one process, which both evaluates the condition and unfolds the computation accordingly. On the contrary, in a choreography, one of the participants must act as a point of control⁸ and notify the involved peers to coordinate the unfolding of the overall interaction.

In our example, we have the role that evaluates the condition (the *Client*) inform with mutually-exclusive channel actions (*dock* or *lookupShip*) the other role of which branch the distributed computation should proceed. Alternatively, one

⁸ There exist consensus protocols used to reach a distributed agreement among a network of peers, but we can see these as ways to elect a leader that takes and communicates a decision to the rest of the peers.

could omit how the two roles coordinate on branch selection and leave that part of the coordination as an implementation detail to the developers. While this choice provides more freedom to the specification language, it leaves a heavy burden on the shoulders of developers of the specification, who must find an agreement (outside what the choreography specifies) on how to implement the piece of coordination left unspecified—e.g., they can agree on using auxiliary channels or extensions of the specified ones to support the coordination.

Returning to our example, depending on the content of the `action` variable of the `Client`, the distributed computation proceeds with either the `dock` or the `lookupShip` operations on channel `chan`. In the first case, we only have one instruction, represented by the interaction `Client.ship -> Harbour.ship` which indicates that the client sends the content of its `ship` variable to the `Harbour`—the latter stores the received data in its local variable `ship`. In the second case, we have the invocation of the `lookupShip` operation by the `Client` (passing the string `name`). Since this is a request-response, our pseudo-language makes it so that the `channel` supports the response as a dual action to the request, marked by the `lookupShip_` label. We find, here, another branching behaviour; this time controlled by a `try-catch` statement. When the `findByName` operation successfully retrieves the `ship` value, the `Harbour` returns it to the `Client`. However, if the instruction fails, the value sent by the `Harbour` is a `ShipNotDocked` fault. Notice that, contrarily to the previous example, the `Harbour` does not govern the branching with the label (of the operation) sent to the `Client`; the interaction happens on the same operation (the response to the `lookupShip` operation) but on different data types—while this difference might seem only at a conceptual level, this can have concrete repercussions on the code of the services that implement a choreography, e.g., as shown in the paragraph on choreography-as-programs.

Closing on choreographies-as-specifications, remind that, while quite concrete, under this interpretation, the choreography shown above is intended as a description of the expected behaviour of the distributed program. The programmers can refer to that specification to manually develop the services that implement the `Client` and the `Harbour`.

As an additional reference to help illustrate the use of WS-CDL as a specification language, we mention the Business Process Modelling Notation (BPMN) Choreography Diagrams, which is a graphical language introduced to model business processes that supports the representation of the interactions among the elements of a model as a choreography. We report in Fig. 15.2 a possible BPMN choreography diagram of the `Harbour` choreography we illustrated above. In the picture, the circles on respectively the left and right sides represent the beginning and ends of the interactions. The diamond is a choice and the boxes represent message exchange, where at the top we find the sender, in the middle the purpose of the message, and at the bottom the receiver.

Choreographies as Types

The interpretation of choreographies as types still considers choreographies as a kind of specification of what and when interactions happen (and what is the shape of the

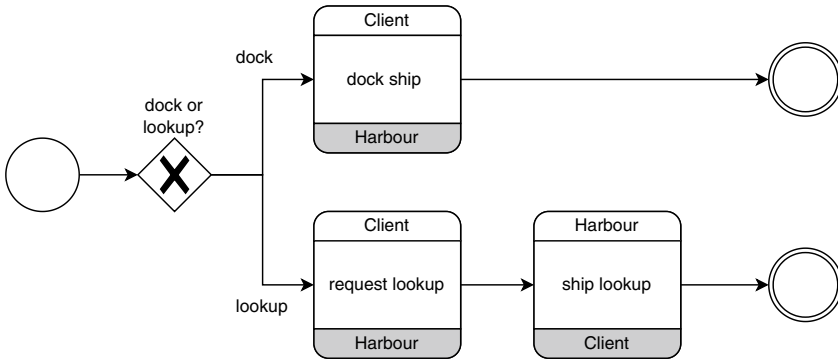


Fig. 15.2 A BPMN representation of the Harbour choreography

data that travels therein) in a given distributed program. However, like other typing disciplines, one can use choreographies-as-types to formally verify the compliance of the programs that implement the roles of a given choreography.

The main example of this interpretation is that of *global session types*; a kind of types used to define and constrain the observable behaviour of processes.

As an example, the following is a possible global session type of the choreography of the previous paragraph.

```
Client -> Harbour: {
  dock( ship ),

  lookupShip( string );
  Harbour -> Client: lookupShip_(ship | ShipNotDocked)
}
```

In the code above, the block `: { ... }` indicates a choice among the comma-separated branches found within the block. For instance, the first interaction, from the `Client` to the `Harbour`, indicates that the `Client` can choose to either invoke the `Harbour` on the `dock` or the `lookupShip` operations. In the first case, `dock`, the interaction stops immediately; in the second, `lookupShip`, the interaction continues with the `Harbour` that either communicates the successful retrieval of the `ship` or a `ShipNotDocked` fault over the same (response) operation, `lookupShip_` (where we drop the brackets after the `:` since there is only one branch).

Given the global session type above, we can use a synthesis procedure that generates a *local session type* for each role. This procedure is usually called a *projection*, since we are projecting the choreographic information that only concerns a specific role, and we can use the resulting local session types to check the correct implementation of a given role.

Among the checks choreographies-as-types can perform, there is the detection of incorrect implementations that can introduce *communication deadlocks*. This is one of the major features of using choreographies—either as types or as programs, as we see also in the next paragraph.

The principle is that, in choreographies, one cannot write a deadlocked communication because send and receive actions are always paired in the atomic interaction “A → B”. This implies that we cannot, e.g., write a choreography where A indefinitely waits for a send action from B and vice versa. Since choreographies cannot introduce communication deadlocks, when we use semantic-preserving projection procedures (i.e., here, such that the checks defined in the global session type carry to the generated local session types), we can use the projected local session types to check the correctness (e.g., communication deadlock-freedom) of the implementations of the roles in the choreography.

Completing our example, the respective synthesised local session types of the `Client` and the `Harbour` could look like the following (shown side-by-side to ease their comparison).

<pre>/* Client local session type */ ! { dock(ship), lookupShip(string); ?lookupShip_(ship ShipNotDocked) }</pre>	<pre>/* Harbour local session type */ ? { dock(ship), lookupShip(string); !lookupShip_(ship ShipNotDocked) }</pre>
--	---

As expected, the generated local session types do not carry atomic, global-level interactions like global session types, but they rather show send and receive actions. Specifically, the `! { ... }` and `? { ... }` blocks respectively indicate the transmission and the reception of a message on one of the operations at the top of the comma-separated branches within the block (we omit the brackets when only one operation is available). Looking at the resulting local session types, we notice how they are one the dual of the other—as the reader might have realised, this kind of duality characterises only binary (i.e., participated by two parties) choreographies, like the one in our example.

Once we obtained the respective types of the `Harbour` and the `Client`, we can check that a given implementation of the two roles correctly follows the observable behaviour expected by the (global) type—we invite the reader to compare the `Harbour` local type with the `Harbour` Jolie implementation in Sect. 15.2.5, respectively mapping successful and faulty responses to the `ship` and `ShipNotDocked` types.

Choreographies as Programs

The interpretation of choreographies as programs postulates that choreographies carry the code that each role shall execute, and a projection procedure generates programs that implement the roles in the choreography.

This approach, also called *choreographic programming*, mainly consists in pairing a fairly concrete choreographic language⁹ with a compiler able to validate chore-

⁹ As a reference, closer to the one we used in the choreographies-as-specifications example rather than the more abstract one of choreographies-as-types.

ographies¹⁰ and project the valid ones into locally-executable programs. Each of these programs implements the semantics of one of the roles in the choreography and thus, when run in parallel, they enact the semantics of their source choreographic program.

Summarising, while choreographies-as-specifications see choreographies as references for implementations, both choreographies-as-types and choreographies-as-programs concern automatic procedures to help implement correct distributed programs. Choreographies-as-types are artefacts that one uses to check the correctness of a given implementation of a role in a choreography. In choreographic programming, a choreography is a “global” program that one can compile into a set of local ones (one for each role in the choreography), which, run together, faithfully execute the semantics of their source choreography.

Considering the relatively concrete language we used in the choreographies-as-specifications example (e.g., with variables and explicit conditional constructs), we re-use it here by interpreting it as a programming language.¹¹ Since we already commented on the salient traits of the choreographic language in the previous paragraphs, here, we concentrate on the other relevant piece of the choreography-as-programs interpretation: the projection to local programs. To illustrate it, we show, side-by-side, the compilation of the example choreography into two possible Jolie services implementing the Client and the Harbour (as the reader might expect, the generated Harbour service is close to the one we described in Sect. 15.2.5).

To proceed with the example, we assume that the `channels` in the choreography have a bit more information, related to the concrete data formats, protocols, and bindings necessary to establish an actual connection between the generated services, so that we can project the related information into Jolie ports.¹²

<pre> /* Client service*/ type ship { /* ... */ } interface chanInterface { oneWay: dock(ship) requestResponse: /* ... */ } outputPort chanHarbour { /* ... */ interfaces: chanInterface } main { /* ... */ if (action == "dock"){ dock@chanHarbour(ship) } else { lookupShip@chanHarbour(name)(ship) } } </pre>	<pre> /* Harbour service*/ /* type ship, chanInterface */ outputPort LocalHarbour { /* ... */ } inputPort chanClient { /* ... */ interfaces: chanInterface } main { /* ... */ [dock(ship)] [lookupShip(name)(ship){ install (ShipNotDocked => { throw ShipNotDocked }) findByName@LocalHarbour(name)(ship) }] } </pre>
--	---

¹⁰ For example, the compiler shall report ambiguous situations where the user needs to specify how to solve knowledge-of-choice situations.

¹¹ As a matter of fact, projects like CHOREOS and CHOReVOLUTION interpret BPMN as a programming language to synthesise local programs that control service interactions.

¹² Alternatively, one can leave to the programmers who obtain the compiled programs the task to fill in the information needed to establish the connection. This detail constitutes a trade-off between interoperability (letting the programmers define the actual bindings of the services) and coupling (fixing in the choreography the connections among the participants).

Above, we see the projection of the Client on the left and that of the Harbour on the right. Both have the necessary `type` and `interface` definitions (referenced in the top comment of the Harbour to ease the line-by-line comparison between the two services). The `channel` from the original choreography is split in `inputPorts` and `outputPorts` owned by the respective services. Specifically, here, we use a rather advanced analyser that can infer that all interactions of channel `chan` in the choreography go from the Client to the Harbour and synthesise only one `outputPort` (`chanHarbour`) for the former and the one `inputPort` (`chanClient`) for the latter (remember that the only interactions from the Harbour to the Client are responses, marked with an underline in the choreography). The Harbour service has an additional port, `LocalHarbour`, which the programmer shall bind to a sub-service that implements the `findByName` operation (e.g., in Sect. 15.2.5, we called this service `Ships`).

In the behaviours of the services, we find (in a way similar to the projection of the types seen in the previous paragraph) the send and receive actions related to the single services. For instance, since it is the Client that evaluates the condition (on its local variable `action`), we find the conditional `if ( action ... )` present in the behaviour of the Client and not in that of the Harbour. Dually, since it is the Client that takes the decision on which branch to go, the Harbour presents a guarded choice, which allows the service to consume any of the possible choices made by the Client. This kind of split between who controls the condition and who must be informed of its evaluation continues with the try-catch construct thereafter. In this case, it is the Harbour that performs an action that can generate that observable and, thus, we find the Jolie-corresponding `install` statement only in the Harbour, where it is used to generate the `ShipNotDocked` fault expected in case of a faulty execution of the `lookupShip` operation.

Notwithstanding its service-oriented origins, recent works on choreographies-as-programs confirmed that the paradigm is not strictly related to service orientation, but one can successfully marry it with other programming paradigms, such as object orientation. The Choral language is a concrete example of this possibility. There, choreographies are objects—the language adopts a syntax that is essentially an extension of that of Java—which can belong to different roles. An interesting result of combining object orientation and choreographic programming is that interactions, which are normally primitives of the language (the “A → B” instructions seen until now), are regular objects owned by two roles that can exchange data using their methods. As with any choreographic programming language, also Choral comes with a compiler which, for example, can compile a Choral program into a set of Java programs that implement the distributed logic of their source choreography.

15.3 Service-Oriented Architectures

In this chapter, we mainly focussed on the linguistic side of service-oriented programming, but we would not provide a complete picture to the reader if we did not mention the other important half of the service-oriented cosmos, which is that of *architectures*.

Of course, giving a comprehensive treatment of service-oriented architectures would require a book on its own (as there are many published just on this matter). In the following, we report a brief comment on the early and recent traits of service-oriented architectures. To do this, we continue to keep the focus that guided our exploration of service-oriented programming and describe how the principles of loose coupling and interoperability impacted and guided the development of the architectural side of the paradigm.

15.3.1 The First Generation: Web Services

Returning to the historical steps from the introduction of this chapter, we mentioned how the advent of the Internet gave a new impulse to distributed programming, generating advanced forms of cooperation between the exponentially-growing number of machines that started accessing the Web.

Since the early 2000s, the service-oriented paradigm gained more and more momentum, allowing developers to create new services either by introducing independent modules in the system or by programming ones that (dynamically) compose the existing services. For instance, one could combine the online services of an airline company, a hotel chain, and a car rental company to create a service that plans a trip. This way of composing services in hierarchies of more and more complex structures took the name of *Service-Oriented Architectures* (SOAs).

From a technological perspective, the at-the-time standard way to build service-oriented programs relied on the realisation of applications that used the so-called *Web Services* (WS) standards: XML, SOAP, WSDL, and UDDI—the latter is a standard for defining service discovery, i.e., which service from an available pool a client should contact, given the type of data requested by the latter. Besides this technology stack, the other distinctive element of first-generation SOAs was the use of an Enterprise Service Bus (ESB, cf. Sect. 15.2.3), which served both as the middleware for routing requests after discovery and mediating protocol integration.

While first-generation SOAs pioneered the paradigm and helped tackle the coupling and interoperability issues of remote procedure calls and stubs (cf. Sect. 14.5.4), they posed one main problem, which prevented their wide adoption—although WS SOAs are still a standard in some middle and large enterprise IT infrastructures. This problem regards the high complexity of setting up, managing, developing, and evolving WS SOAs, both from the point of view of (knowing and deploying) the technology stack and of the design, evolution, and code reuse of the services themselves.

On the one hand, approaching WS SOA development appears as a daunting task from the get-go. Indeed, as mentioned earlier, one can start developing WS SOAs with the combination of WSDL and SOAP (and a programming language of choice). However, the full technology stack of web services results in a complex set of standards—the cited WSDL, SOAP, etc., but also many more, of which we report a non-exhaustive schematisation in Fig. 15.3 (and leave the task of looking up their definition and purposes to the daring reader)—which can easily scare away both

Business processes	WSFL, BPEL4WS, WS-CDL, WS-CAF		
Quality of Service	WS-SecureConversation, WS-SecurityPolicy, WS-Trust	WS-Coordination, WS-Transaction, WS-AtomicTransaction, WS-BusinessActivity	WSDM, WS-Manageability, SPML, WS-Provisioning
Description and Discovery	UDDI, WSIL, WS-Discovery		
Messaging	WSDL, WSCL, WSCI, WS-MetadataExchange, WS-Policy		
	SOAP, WS-Addressing, WS-Notification, WS-Eventing, WS-Enumeration, WS-MessageDelivery, WS-Reliability, WS-ReliableMessaging, WS-Resource, WS-Transfer		
	XML, HTTP, SMTP, FTP, etc.		

Fig. 15.3 Schematisation of a (non-exhaustive) list of WS standards

programmers who are approaching the paradigm and companies that would need to commit money and time on technology and know-how to run their IT systems. This complexity factor is further exacerbated by the ubiquitous presence of ESB technologies. Indeed, while ESBs make it easy to connect and let services interact, the fact that they can host some logic for mediating the communication among services leads programmers to put service-specific code there rather than further complicating the existing services. Paradoxically, in this way, ESBs make SOAs more complex (since now the actual behaviour of a service is fragmented in parts of the ESB), and they increase the coupling and limit the interoperability of services, since the ESB becomes an essential component for the correct execution the SOA.

On the other hand, the high complexity of setting up and publishing a service makes it easier to build “big” services—dubbed *monoliths* in recent years—that implement disparate functionalities, rather than follow the established principle of separation of concerns and develop a service for each set of cohesive operations.¹³ The hindering aspect of this tendency towards monoliths regards the (again) difficulty in managing, evolving, and reusing services. Indeed, larger and sparser codebases require the developer to keep in mind and reason on an equally large and intricate network of interacting components. Moreover, it is more difficult to extract and reuse subsets of the (implementations of) operations, because these tend to share components therein (e.g., imagine different operations that reuse the same data structures and modules). These pitfalls—and the lack of guiding principles to avoid them—further dissuaded developers from developing their distributed programs as SOAs.

¹³ As an example, we can classify a service as a monolith if it implements the operation for, e.g., doing groceries, accounting, and warehouse management. While these are somewhat related (e.g., they all keep track of resources), their application domains (and data types, behaviours, etc.) are different and belong in separate modules.

Lightweight Standards and Cohesive Interfaces

The problems listed above pushed for a new set of architectural principles that introduced the second generation of service-oriented architectures.

The first principle is that of using lightweight standards for developing services. This is a direct response to the frightening scenario depicted in Fig. 15.3, and, more practically, to the still-high complexity of the “baseline” technologies, like WSDL and SOAP, whose irksomeness was further exasperated by the severe verbosity and undecipherability of the XML format. The most renowned example of this shift to lightweight standards is the mentioned REST style (see Sect. 15.2.4).

The second principle regards the design of service interfaces, it is called *cohesion*, and it has been introduced to contrast the uncontrolled growth of contextually-sparse services and to ease code reuse. To understand cohesion, let us consider an example of a service that provides two operations, internally implemented by two submodules: the `login` operation, which lets users access the other operations of the service, and the `compute` operation, which performs some heavy-weight computations upon request by logged users. In particular, since the operations belong in the same service, their sub-modules share some internal data to keep the state of logged users—e.g., `login` creates a token that it both sends to the authenticated user and saves in a shared data structure that `compute` can access, to validate user requests that carry said token. The fact that the service provides this diverse set of functionalities makes it uncohesive, where cohesion indicates that a service (or, in general, a software unit) implements only functionalities strongly related to the concern that it is meant to deal with. We can contrast this uncohesive, monolithic service with an architecture made of two services, one offering the `login` operation and the other the `compute` one. Having cohesive services helps to achieve (efficient) *code reuse*. Indeed, if we wanted to build our distributed program with a new set of functionalities, we could do it by adding dedicated units and reuse what the program already offers, e.g., we would not need the new services to handle user authentication, since they can delegate these tasks to the existing “login” unit (the one implementing the `login` operation). On the contrary, in the monolith, the `login` and `compute` operations are integrated via internal memory and, unless we employ some (fragile) workaround to, e.g., intercept the token and share it with the new units, we cannot reuse the `login` functionality for other services. Similarly, our architecture becomes more modular, so that, if we did not need to have logged users (e.g., because we are in a trusted network), we could just run the “compute” service. We can apply the same principle when building new service architectures: if we have to handle user authentication, we do not necessarily need to start from scratch and rather use a copy of the service we previously built. Notably, having cohesive services spares us from integrating functionalities that are inessential to our program. For instance, imagine using the monolith in the example to just obtain the functionalities provided by the `login` operation. In doing so, we compromise the performance and quality of our distributed program. For example, we waste resources (CPU, memory) to provide the unnecessary `compute` operation and malicious users could exploit it to overload the service by flooding the service with `compute` invocations (performing a denial-of-service attack).

In a sense, we can see cohesiveness as an interpretation of the principle of loose coupling, applied at the functionality level. We want functionalities to be as loosely-coupled as possible, so we follow the convention of cohesive interfaces to help us achieve a good balance between spreading the functionalities among the fewest possible services and, at the same time, having units which we can easily add, remove, modify, and reuse.

15.3.2 Second Generation: Microservices and Serverless

The most recent incarnation of SOAs is that of microservice architectures, where the “micro” prefix underlines the tension towards minimising the set of functionalities a service provides to that cohesive set we illustrated in the previous section. As a consequence, microservices tend to abandon (or, more accurately, cherry-pick some useful elements of) the WS technology stack of the first SOA generation to further technology agnosticism. The motto for microservices is that, since the only formal dependency/obligation a microservice has is its interface and how it communicates (usually adopting open standards, e.g., HTTP), the programmer can choose the technologies that best fit their specific application scenario. This impulse towards minimality and independence met the necessity of large teams of developers, who collaborate on the implementation of the same distributed program. Here, microservices proved to be a fitting architectural style. The teams would divide into subgroups, each responsible for the development, evolution, and execution of a specific microservice that makes up the distributed program. The shift to microservices, which started in the 2010s, marked the widespread return of SOAs, with companies like Netflix, Amazon, Facebook, and Twitter as industry spearheaders of the revived architectural style, which is the state of the art of today’s distributed system architectures.

From the language point of view, we can see some concepts from service-oriented programming analysed in the previous parts of this chapter catering for the needs of microservice programming.

For instance, in microservices, we need to reconcile the need for specifying the contracts of the services (concretely, their types and interfaces) and the tension towards technology agnosticism, which dictates the usage of abstract specifications that one can then tie to a specific, concrete technology—of course, these are the principles of loose coupling and interoperability that guided our exploration of the paradigm since the beginning of the chapter. As observed in Sects. 15.2.2 and 15.2.4, languages can help on this matter by providing data-format-abstract types and interface languages that support the definition of abstract, explicit messaging patterns. These abstractions allow the user to define their services without committing to a concrete technology stack, and they leave the grounding to configurations that the user pairs with the target service, depending on the specifics of the given deployment scenario.

Another aspect is that of service composition. As discussed earlier, in SOAs (microservice architectures in particular), services may cooperate to provide more complex and elaborate functionalities. Here, the principle of modularisation of

microservices competes with the complexity of managing a cosmos of software units to achieve a given goal. Using languages based on other paradigms than services (e.g., functions or objects) can work against the user. Indeed, as we illustrated with our Java orchestrators in Sect. 15.2.5, adaptations (e.g., via annotated mappings) quickly fall short when we raise the complexity of coordination. On the contrary, service-oriented languages can offer great support to developers in defining advanced behaviours. To build advanced cooperation logic, we mentioned the existence of service-oriented languages based on the complementary approaches of orchestration and choreography (cf. Sect. 15.2.1).

To complete the picture of second-generation SOAs, we need to take a quick detour on the state of the art in distributed system deployment, based on “cloud computing”. The introduction of service orientation occurred at around the same time as the rise of cloud computing and its “as-a-service” business model.¹⁴ In this model, cloud providers offer storage (disk space, databases), networking (network topologies, communication services), and computation as commodities, which developers can access flexibly and on-demand. In practice, instead of requiring developers to buy hardware and manage execution platforms and software, they can easily assemble (as well as release) these components through the services offered by some cloud providers. The business, from the side of cloud providers, regards resource sharing: when a customer requires some computation resources or some storage, the provider would not buy some new machine, set it up, and give exclusive access to the customer. On the contrary, cloud computing technologies allow the provider to share the multitude of physical computers (the “cloud”) it owns and manages, by partitioning and assembling their computational, storage, and networking resources into virtual machines dedicated to its customers.

A recent addition to the family of cloud computing services is the execution model called “serverless” and, in particular, that of “Functions-as-a-Service” (in the following, we use the two terms interchangeably). In serverless, the developer writes some functions—looking at service orientation, imagine a function as a service providing only a single operation—and asks the cloud provider to execute them in response to some trigger (e.g., in response to some request). The developer does not manage any aspect of the infrastructure or platform that runs its function (i.e., they do not manage the server where its function runs, hence the misnomer “serverless”, although servers are, of course, involved). Every time the trigger activates, the provider puts a new instance of the function in execution. In essence, serverless computing dispenses the developer from assembling the infrastructure that runs its architecture of functions. The focus on functions loosens the concept of cohesiveness seen in microservices: since functions are single behavioural units that run independently, they are not bound within some software module (e.g., a microservice) that compels developers in designing and developing connected functionalities in cohesive interfaces.

¹⁴ As a proof of the contemporaneity of and interplay between first-generation SOAs and cloud computing, one of the earliest cloud providers, Amazon, named their subsidiary “Amazon Web Services”.

We can see the two styles of microservices and serverless as dual. In particular, serverless functions give more freedom to developers—in a way, bringing to an extreme the principles of loose coupling and interoperability—but they also present new challenges for developers due to the high fragmentation of their composition model, which demands more reasoning and discipline than microservices. Also in this case, (service-oriented) languages can help in harnessing this complexity and support programmers in defining and building correct implementations.

15.4 Summary

In this chapter, we have covered the main aspects of service-oriented programming and briefly looked at the complementary concepts related to service-oriented architectures. Of course, both treatments (the second, in particular) are far from being an exhaustive representation of the matter. Our aim, here, has been to provide the essential linguistic concepts that mark the service-oriented paradigm, and we invite interested readers in deepening their knowledge by consulting books dedicated to either topic.

In this chapter, we focussed on aspects of service-oriented programming:

- the distinctive traits of distributed programs, the need for open systems, and the introduction of the principles of loose coupling and interoperability, which characterises the origin and evolution of the service-oriented programming paradigm;
- services, explained following a concept-based structure that breaks them down into their fundamental elements: data types, interfaces, and behaviours. For each element, we looked at language constructs introduced to address the problems of composing a program made of distributed units and, in particular, we scrutinised these concerning their support for loose coupling and interoperability;
- service-oriented architectures, which are an important driving force that influenced the evolution of service-oriented programming. The study of their trends and phases helps us understand the motivations behind the introduction, adoption, and obsolescence of some language constructs that characterise service-oriented programming.

15.5 Bibliographical Note

The literature on service-oriented programming is vast and covers a wide range of topics, from theoretical concepts to practical guides. As an offshoot of concurrent/distributed programming, also service orientation has many theoretical contributions, e.g., coming from the tradition of process algebras and focussed on the formalisation of services (data types, interfaces, and behaviours), and the verification of their prop-

erties. In addition to the fundamental texts, on concurrency, mentioned in Sect. 14.9, literature on service-oriented programming includes models for service orchestration [1,2] and choreographies [3–7]. Literature on the Jolie programming language includes its foundational theory [8], its conceptual and programming models [9] and many applications, see [10].

Besides languages, service orientation also has a broad literature on architectures, with instructional books on modelling and designing first- and second-generation SOAs [11–14].

15.6 Exercises

1. You want to build an `articles` service, part of a blog distributed architecture. You start from the data structures it must handle. Make some examples, using the XML and JSON formats, of article values that hold the following information: article identifier (e.g., an integer), title of the article, list of authors, date of publication, text of the article. Define a JSON schema that validates any viable instantiation of the article values that you exemplified. Define a Jolie type for the same purpose. Spot the relevant differences and similarities between the two ways of specifying data types for services (in particular, considering the open and close stance the two have on subtyping).
2. You want to build a second service, `comments`, to also handle article comments. Walking the same steps you followed at point 1, exemplify and validate comment values, which shall hold: a comment identifier (e.g., an integer, can also be used to temporally order the comments), identifier of its related article, author of the comment, date of the comment, content of the comment.
3. Describe the interfaces of the services that offer the operations to submit and read articles and comments. Namely, the `articles` interface provides two operations: submit a new article and retrieve the last X articles (where X is a number), while the `comments` interface offers the two operations: submit a new comment and retrieve the last X comments (where X is a number), both related to some article Y (where Y is an article identifier). Formalise the two interfaces using WSDL, Jolie interfaces, and the REST format (possibly re-using the structures and validation formats at the previous items).
4. Write an `articles` Java orchestrator that implements the `articles` interface at point 3 as a REST service (e.g., using Java Spring).
5. Write a `comments` Jolie orchestrator that implements the `comments` interface at point 3 as a REST service.
6. Write a `client` service that interacts with the `articles` and the `comments` services to implement an operation that allows users to retrieve the last X (number) articles and their Y (number) related comments. Use the technology you prefer for the implementation of the `client`.
7. Write a `gateway` orchestrator that implements both the `articles` and the `comments` interfaces by forwarding the requests to and the responses back from

- the two namesake services at points 4 and 5. Try to implement two versions of the gateway: one in Jolie and one in Java (Spring). Notice the different ways in which orchestration works under the two alternatives.
8. Modify the `client` at point 6 to interact with the gateway at point 7. Notice the modifications you need to perform on the client to make it interact with the gateway in place of the `articles` and `comments` services. Did you need to substantially change the logic of the application or did the changes remain confined to only a set of parameters? What would happen if you needed to change, e.g., the format of the data from, e.g., JSON to XML, or to switch from REST/HTTP to another protocol, e.g., SOAP? Ponder how the different technologies seen in this chapter help in minimising those modifications.
 9. Define a choreographic specification (e.g., using the pseudo-language presented in Sect. 15.2.5) that implements the operation of the `client` orchestrator at point 6. The choreography has four participants, a `user`, a `client`, an `articles`, and a `comments` services. The latter three implement the interfaces of their namesake services at the previous points. Consider using the choreography you defined as-type (with the appropriate type abstractions) and as-program and observe whether and how the services that you implemented align with the different choreographic artefacts.

References

1. A. Lapadula, R. Pugliese, F. Tiezzi, A calculus for orchestration of web services, in *European Symposium on Programming* (Springer, 2007), pp. 33–47
2. H.T. Vieira, L. Caires, J.C. Seco, The conversation calculus: a model of service-oriented computation, in *European Symposium on Programming* (Springer, 2008), pp. 269–283
3. R. Hamadi, B. Benatallah, A Petri net-based model for web service composition, in *Proceedings of the 14th Australasian database conference*, vol. 17, pp. 191–200 (2003)
4. M. Autili, P. Inverardi, M. Tivoli, Automated synthesis of service choreographies. *IEEE Softw.* **32**(1), 50–57 (2014)
5. D. Ancona, V. Bono, M. Bravetti, J. Campos, G. Castagna, P. Deniérou, S.J. Gay, N. Gesbert, E. Giachino, R. Hu, et al., Behavioral types in programming languages. *Found. Trends® Program. Lang.* **3**(2–3), 95–230 (2016)
6. M. Dalla Preda, M. Gabbriellini, S. Giallorenzo, I. Lanese, J. Mauro, Dynamic choreographies: theory and implementation. *Log. Methods Comput. Sci.* **13**(2) (2017)
7. F. Montesi, *Introduction to Choreographies* (Cambridge University Press, 2023)
8. C. Guidi, R. Lucchi, R. Gorrieri, N. Busi, G. Zavattaro, Sock: a calculus for service oriented computing, in *International Conference on Service-Oriented Computing* (Springer, 2006), pp. 327–338
9. F. Montesi, C. Guidi, G. Zavattaro, Service-oriented programming with Jolie, in *Web Services Foundations* (Springer, 2014), pp. 81–107
10. The Jolie team. Jolie home page (2014). <https://www.jolie-lang.org/>. Accessed 05 Jan. 2023
11. G. Hohpe, B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions* (Addison-Wesley Professional, 2004)
12. T. Erl. *Service-Oriented Architecture: Concepts, Technology, and Design* (Pearson Education, 2005)

13. S. Newman. *Building Microservices: Designing Fine-Grained Systems* (O'Reilly Media, 2015)
14. O. Zimmermann, M. Stocker, D. Lübke, U. Zdun, C. Pautasso. *Patterns for API Design-Simplifying Integration with Loosely Coupled Message Exchanges* (Addison-Wesley Professional, 2022)